Análisis en Lean 4

Alex Kontorovich

Traducido por: José A. Alonso

Sevilla, 11 de mayo de 2026

Índice general

1. La historia del análisis real **4**

1.1. Preámbulo 4

1.2. Introducción a Lean 11

1.3. La táctica `rfl` 12

1.4. La táctica `rw` 13

1.5. La táctica `ring_nf` 14

1.6. La táctica `use` 15

1.7. La táctica `intro` 15

1.8. La táctica `specialize` 16

1.9. El desafío táctico definitivo 19

2. El cálculo de Newton del número pi **22**

2.1. La revolución matemática de 1666 . . . 22

2.2. La definición principal 36

3. Convergencia de sucesiones **48**

3.1. Más sobre sucesiones 48

3.2. Nivel 1: La propiedad arquimediana . . 48

3.2.1. La estrategia de demostración en lenguaje natural 49

3.2.2. Los desafíos de la implementación en Lean 50
3.2.3. Nuevas herramientas que necesitarás 52
3.2.4. Demostraciones en Lean 52
3.2.5. Demostración en lenguaje natural de la propiedad arquimediana 55
3.2.6. Revisión de errores comunes . 56
3.3. Nivel 2: Nuestro primer límite real . . . 57
3.3.1. El objetivo: Demostrar que $1/n \to 0$ 57
3.3.2. Recordatorio: La definición de convergencia de sucesiones . . 57
3.3.3. Estrategia de demostración en lenguaje natural 58
3.3.4. Los desafíos de la implementación en Lean 59
3.3.4.1. Desafío 1: Multiplicación cruzada de fracciones 59
3.3.4.2. Desafío 2: Aritmética lineal 60
3.3.4.3. Desafío 3: Casting explícito de tipos 60

3.3.4.4. Desafío 4: Casting en aplicaciones de tácticas 60
3.3.5. Nuevas herramientas que necesitarás 61
3.3.6. Consejos profesionales para este nivel 61
3.3.7. Demostración en Lean 62
3.3.8. Lo que acabamos de lograr . . . 66

4. No convergencia **68**
4.1. Un gran debate 68
4.2. Paso 2: Solución en Lean 73
4.3. Paso 3: Demostración en lenguaje natural 80
4.3.1. Demostración en lenguaje natural 80
4.3.2. Ideas clave 81

5. Duplicar una sucesión convergente **82**
5.1. El desafío de escalar en la fábrica . . . 82
5.2. La configuración matemática 83
5.2.1. Idea Clave: Escalado Inverso de la Tolerancia 83
5.3. Nuevas herramientas que necesitarás . 84

5.3.1. Valor absoluto de productos . . 84
5.4. Enfoque estratégico 84
5.5. Solución en Lean 85
5.6. Prueba en lenguaje natural 90
5.6.1. Enunciado del teorema 91
5.6.2. Prueba 91
5.7. Por qué importa este patrón 92
5.7.1. La estrategia elegante 93

6. Unicidad y teoremas avanzados de límites 94
6.1. Unicidad de límites 94
6.1.1. Nuevas herramientas 95
6.1.1.1. Prueba por contradicción: `by_contra` . . . 95
6.1.1.2. Positividad del valor absoluto: `abs_pos_of_nonzero`
6.1.2. Enunciado matemático 95
6.1.3. Demostración en Lean 95
6.1.4. Prueba en lenguaje natural . . . 96
6.1.5. Por qué importa la unicidad . . 97
6.2. Eventualmente: Las sucesiones convergentes permanecen cerca de su límite . 97
6.2.1. Enunciado matemático 98

6.2.2. Enfoque estratégico 98
6.2.3. Demostración en Lean 99
6.2.4. Prueba en lenguaje natural . . . 99
6.2.5. Aplicaciones 100
6.3. Continuidad del valor absoluto: Sucesiones de valores absolutos 100
6.3.1. Nuevas herramientas 101
6.3.1.1. Propiedad de Lipschitz: `abs_Lipschitz` 101
6.3.2. Enunciado matemático 101
6.3.3. Demostración en Lean 101
6.3.4. Prueba en lenguaje natural . . . 102
6.3.5. Por qué importa este resultado 102
6.4. Recíprocos de sucesiones convergentes 103
6.4.1. Nuevas herramientas 103
6.4.1.1. Valor absoluto de cocientes: `abs_div` . . . 103
6.4.1.2. No nulo a partir de valor absoluto positivo: `nonzero_of_abs_pos` 104
6.4.2. Enunciado matemático 104
6.4.3. Enfoque estratégico 104
6.4.4. Demostración en Lean 105

6.4.5. Prueba en lenguaje natural . . . 106
6.4.6. Prueba conseguida 107
6.4.7. Lo que lograste 107
6.4.8. Aplicaciones y extensiones . . . 108
6.4.9. Dominio de la técnica 109

7. Teoremas avanzados de límites e inducción 110

7.1. Introducción: Inducción Matemática . . 110
7.2. Jefe Final: Recíprocos de Sucesiones Convergentes 115
7.2.1. El Marco Matemático 115
7.2.2. Nuevas Herramientas 115
7.2.3. Enfoque Estratégico 116
7.2.4. Solución en Lean 116
7.2.5. Demostración en Lenguaje Natural 118
7.2.6. Aplicaciones y Extensiones . . 119
7.3. Por Casos: Análisis de Casos Sin Hipótesis 120
7.3.1. La Táctica `by_cases` 120
7.3.2. Aplicación: Eventualmente Mayor que la Mitad del Límite . . 121
7.3.3. Solución en Lean 121

7.3.4. Demostración en Lenguaje Natural 122
7.3.5. Importancia 122
7.4. Inducción Matemática 123
7.4.1. El Principio de Inducción Matemática 123
7.4.2. La Táctica `induction'` 123
7.4.3. Nueva Herramienta: `ge_one_of_nonzero` 1
7.4.4. Ejemplo: Crecimiento Exponencial 124
7.4.5. Solución en Lean 124
7.4.6. Demostración en Lenguaje Natural 125
7.4.7. Aplicaciones 125
7.4.8. Idea Clave 126

1. La historia del análisis real

1.1. Preámbulo

SIMPLICIO: ¿Qué es el "Análisis real"?

SÓCRATES: Oh, es simplemente Cálculo, pero "bien" hecho.

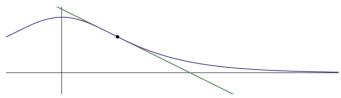
SIMPLICIO: ¿Eh? ¿Por qué el Cálculo necesita un nombre nuevo? ¿Qué tiene de malo?

SÓCRATES: Bueno, nada en realidad. Rápido: ¿qué es una derivada?

SIMPLICIO: ¡Fácil! Si tengo una función $f : \mathbb{R} \to \mathbb{R}$ y es derivable en x, entonces la derivada es:

$$f'(x) := \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

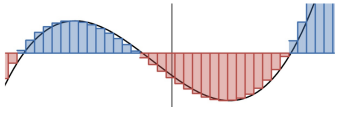
Esto representa la pendiente "instantánea" de la gráfica de la función $y = f(x)$ en el punto $(x, f(x))$.



SÓCRATES: ¡Muy bien! Y dime por favor, ¿qué es una integral?

SIMPLICIO: ¡Eso también es fácil! Si quieres integrar nuestra función f a lo largo de un intervalo, $[a, b]$, por ejemplo, finges que tienes infinitos rectángulos infinitamente pequeños, calculas sus áreas como base por altura y los sumas:

$$\int_a^b f(x)dx := \lim_{N\to\infty} \sum_{j=1}^{N} \frac{b-a}{N} f\left(a + j\frac{b-a}{N}\right)$$



SÓCRATES: Estupendo. ¿Y cuáles son los dos Teoremas Fundamentales del Cálculo?

SIMPLICIO: ¡Estos también son fáciles! El primero dice que si creas una nueva función integrando f hasta una cantidad variable, x, es decir, defines:

$$F(x) := \int_a^x f(t)dt$$

entonces la derivada de la nueva función es simplemente $F'(x) = f(x)$.

SÓCRATES: ¿Y el segundo?

SIMPLICIO: El segundo dice que, a la inversa, si F es una primitiva de f, es decir, $F'(x) = f(x)$, entonces es fácil calcular el área bajo la curva, porque:

$$\int_a^b f(x)dx = F(b) - F(a)$$

¡Así que la derivación y la integración son operaciones inversas!

SÓCRATES: Perfecto. Ahora, aquí está el problema. Has usado palabras como "límite", "infinitos", "infinitamente pequeño", y así sucesivamente. ¿Qué significan *realmente*?

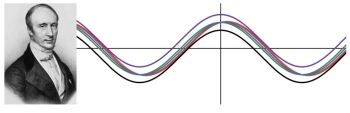
SIMPLICIO: Oh, ya sabes, es cuando algo sucede "eventualmente". Solo tienes que acostumbrarte a la sensación.

SÓCRATES: Hmm sí, entiendo. Estoy de acuerdo en que esa es una forma aceptable de pensarlo, al menos por un tiempo, y una que le iba bien a Newton (que se sentía bastante incómodo con tales palabras) y a Leibniz (que era más despreocupado en este aspecto), los dos inventores del cálculo del siglo XVII (si no contamos a personas como los antiguos griegos Eudoxo y Arquímedes, o al indio Madhava del siglo XIV... pero esto no es una lección de historia). Leibniz enseñó a los hermanos Bernoulli (¡los "primeros estudiantes de bachillerato tecnológico" del mundo!), quienes enseñaron, entre otros, al marqués de l'Hôpital y al gran Leonhard Euler (el primer "nativo del cálculo"), quien nos enseñó al resto de nosotros. Todo iba bastante bien... y entonces llegó el siglo XIX.



SIMPLICIO: ¿Eh? ¿Qué pasó en el siglo XIX?

SÓCRATES: Verás, un tipo llamado Augustin-Louis Cauchy apareció (más o menos en la década de 1810) y empezó a protestar porque no estábamos haciendo las cosas de forma perfectamente "rigurosa". Se propuso volver a demostrar los teoremas del cálculo usando definiciones precisas en lugar de explicaciones vagas. Estaba haciendo grandes progresos, incluyendo la demostración de afirmaciones como: el límite de funciones continuas es continuo.



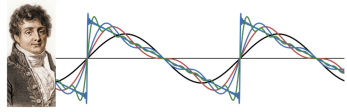
SIMPLICIO: Claro, eso suena perfectamente razonable. Un límite es un proceso continuo, así que si se lo aplicas a funciones continuas, al final deberías obtener algo continuo también. ¿No?

SÓCRATES: Bueno, el problema es que por la misma época, un tipo llamado Joseph Fourier andaba por ahí afirmando que podía sumar un montón de senos y cosenos y obtener básicamente cualquier función que quisiera, ¡incluyendo, por ejemplo, la función discontinua de diente de sierra!

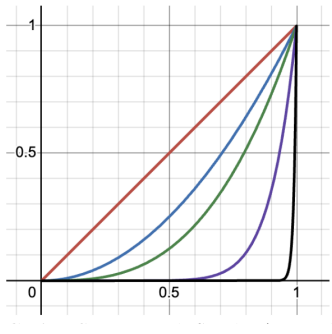
SIMPLICIO: ¡¿Qué?!

SÓCRATES: Mira tú mismo: aquí tienes una gráfica de $\sum_{n=1}^{100} \frac{1}{n} \sin(nx)$. A medida que llevas el 100 al infinito, ¡Fourier afirma que esto se acercará cada vez más

a una función de diente de sierra!



SIMPLICIO: ¡Hala! Espera, se me ocurre un ejemplo aún más fácil: solo hay que mirar la familia más simple de polinomios, $f_n(x) = x^n$, en el intervalo unidad $[0, 1]$. Cuando tomas potencias altas de cualquier punto estrictamente menor que 1, eso tiende a 0 en el límite, pero las potencias de 1 siempre se mantienen en 1. ¡Así que la función límite también es discontinua! ¿Qué demonios está pasando aquí?



SÓCRATES: ¡Muy bien, Simplicio! Exactamente, entre Fourier y Cauchy, "rompieron las matemáticas". ¡El que lo rompe, lo paga!

SIMPLICIO: Vale, entonces ¿cuál es la respuesta correcta? ¿Cómo se hace el cálculo con rigor?

SÓCRATES: ¡No tan rápido! Las cosas se pusieron aún peor, y a mediados del siglo XIX, ¡la gente se dio cuenta de que ni siquiera sabíamos qué *son* los números reales!

SIMPLICIO: ¿Qué? ¿Cómo que qué son? Aquí los tienes: está el cero, y el uno, y el -2, y $3/5$, y $\sqrt{2}$, y e y π. ¿Cuál es el problema?



SÓCRATES: Bueno, ¿recuerdas que en cálculo necesitas algo llamado el teorema del valor intermedio?

SIMPLICIO: Claro, si tienes una función continua y pasa de ser negativa a ser positiva, entonces tiene que cruzar el cero en algún punto intermedio.

SÓCRATES: Muy bien. Háblame de la función $f : x \mapsto x^2 - 2$. (Decimos: "f asigna x a $x^2 - 2$". Observa que escribimos $f : \mathbb{R} \to \mathbb{R}$ para indicar que f toma números reales y devuelve números reales, pero sustituimos $\to$ por $\mapsto$ cuando queremos decir qué le ocurre a una entrada particular x bajo la acción de f). En particular, ¿qué le ocurre a f en los números racionales?

SIMPLICIO: Vale, bueno, si x es un número racional, también lo es x^2, y por tanto también $x^2 - 2$. Así que, en realidad, podríamos decir que $f : \mathbb{Q} \to \mathbb{Q}$, es decir, f asigna números racionales a números racionales. Sobre los reales, la gráfica de $y = f(x)$ es una parábola simple. Pero me habías preguntado por el teorema del valor intermedio. Hmm. Cuando $x = 0$, sé que

$f(0) = 0^2 - 2 = -2$, que es negativo. Y cuando $x = 2$, $f(2) = 2^2 - 2 = 2$, que es positivo.

SÓCRATES: Continúa...

SIMPLICIO: Así que hay una raíz de f en algún lugar entre 0 y 2. Pero el lugar donde f cruza el eje x es en $x = \sqrt{2} \approx 1{,}41\ldots$.

SÓCRATES: ¿Y qué sabían los pitagóricos sobre este número?

SIMPLICIO: Supuestamente uno de ellos, Hipaso, descubrió que $\sqrt{2}$ es irracional, lo que arruinó toda su teoría de la geometría y la forma (originalmente creían que *todos* los números eran racionales); la leyenda dice que Hipaso fue ahogado en el mar por su herejía.

SÓCRATES: Entonces...

SIMPLICIO: ¡¿Entonces espera, si nos limitamos a las entradas racionales, esta parábola es negativa, luego es positiva, y *nunca* cruza el cero?! Pero hay montones de números racionales por todas partes. Entonces, ¿qué hace que los números reales sean diferentes de los números racionales para que el teorema del valor medio sí se cumpla?

SÓCRATES: ¡Ah! Ahora, amigo mío, estamos listos para comenzar.

1.2. Introducción a Lean

Software de demostración de teoremas

En este curso, utilizaremos un "asistente de demostración" llamado Lean. Se trata de un software que comprueba que nuestras demostraciones prueban *exactamente* lo que afirmamos que prueban.

Teorema 1.2.1. *Si* $x = 5$, *entonces* $x = 5$.

```
import Mathlib.Tactic

variable (x : ℝ)

-- 1ª demostración
example
  (h : x = 5)
  : x = 5 :=
by
  exact h

-- 2ª demostración
example
  (h : x = 5)
  : x = 5 :=
by
  apply h

-- 3ª demostración
example
  (h : x = 5)
  : x = 5 :=
by
  assumption
```

Nota: Se escribe `exact h` ya que la hipótesis `h` es exactamente lo que queremos demostrar.

1.3. La táctica `rfl`

Cuando las cosas son idénticas a sí mismas

A veces, en matemáticas, necesitamos demostrar que algo es igual a sí mismo. Por ejemplo, demostrar que $x^2 + 2y = x^2 + 2y$. En Lean, usamos la táctica `rfl` (abreviatura de *reflexivity*).

Teorema 1.3.1. $x^2 + 2y = x^2 + 2y$

```
import Mathlib.Tactic

variable (x y : ℝ)

-- 1ª demostración
example : x ^ 2 + 2 * y = x ^ 2 + 2 * y :=
by
  rfl

-- 2ª demostración
example : x ^ 2 + 2 * y = x ^ 2 + 2 * y :=
by
  simp
```

1.4. La táctica `rw`

Reescribir con igualdades

Supongamos que tienes una hipótesis llamada `Bob : x = 2`, y tu objetivo es demostrar que $x + y = 2 + y$. Como `Bob` nos dice que $x = 2$, podemos sustituir x por 2 usando `rewrite`.

Teorema 1.4.1. *Si $x = 2$, entonces $x + y = 2 + y$.*

```
-- 1ª demostración
example
  (h : x = 2)
  : x + y = 2 + y :=
by
  rewrite [h]
  -- ⊢ 2 + y = 2 + y
  rfl

-- 2ª demostración
example
  (h : x = 2)
  : x + y = 2 + y :=
by
  subst h
  -- ⊢ 2 + y = 2 + y
  rfl

-- 3ª demostración
example
  (h : x = 2)
  : x + y = 2 + y :=
by
  rw [h]
```

```
-- 4ª demostración
example
  (h : x = 2)
  : x + y = 2 + y :=
by
  simp [h]
```

1.5. La táctica ring_nf

Manipulaciones algebraicas

La táctica ring_nf (ring normal form) puede realizar automáticamente expansiones de productos, agrupar términos y aplicar la propiedad distributiva.

Teorema 1.5.1. *El desarrollo binomial:* $(x+y)^3 = x^3 + 3x^2y + 3xy^2 + y^3$.

```
-- 1ª demostración
example : (x + y)^3 = x^3 + 3*x^2*y + 3*x*y^2 + y^3 :=
by
  ring

-- 2ª demostración
example : (x + y)^3 = x^3 + 3*x^2*y + 3*x*y^2 + y^3 :=
by
  ring_nf
```

1.6. La táctica use

Demostrar la existencia

Para demostrar una afirmación existencial (∃), debes proporcionar un valor específico que funcione usando la táctica use.

Teorema 1.6.1. *Existe un número real que hace que este desarrollo binomial funcione*

$$(x+y)^4 = x^4 + 4x^3y + cx^2y^2 + 4xy^3 + y^4$$

```
example : ∃ c, (x + y)^4 = x^4 + 4*x^3*y + c*x^2*y^2 +
 ↪  4*x*y^3 + y^4 :=
by
  use 6
  -- ⊢ (x+y)^4 = x^4 + 4*x^3*y + 6*x^2*y^2 + 4*x*y^3 +
 ↪  y^4
  ring
```

1.7. La táctica intro

Sentencias universales

Para demostrar una afirmación "para todo" (∀), introducimos un elemento arbitrario y las hipótesis asociadas usando intro.

Teorema 1.7.1. *Para todos los números reales positivos, se cumple esta identidad algebraica.*

$$(\epsilon + 1)^2 = (\epsilon + 1)^2$$

```
example : ∀ ε : ℝ, ε > 0 → (ε + 1) ^ 2 = (ε + 1) ^ 2 :=
by
  intro ε h
  -- ε : ℝ
  -- h : ε > 0
  -- ⊢ (ε + 1) ^ 2 = (ε + 1) ^ 2
  rfl
```

1.8. La táctica `specialize`

Uso de sentencias universales

Si tienes un ∀ en una **hipótesis** y quieres aplicarlo a un valor específico, usas `specialize`.

Teorema 1.8.1. *Si una función de x siempre es igual a x^2, entonces es igual a t^2 cuando se evalúa en t.*

```
-- 1ª demostración
example
  (t : ℝ)
  (ht : t > 0)
  (f : ℝ → ℝ)
  (hf : ∀ x > 0, f x = x ^ 2)
  : f t = t ^ 2 :=
by
```

```
  specialize hf t
  -- hf : t > 0 → f t = t ^ 2
  specialize hf ht
  -- hf : f t = t ^ 2
  exact hf

-- 2ª demostración
example
  (t : ℝ)
  (ht : t > 0)
  (f : ℝ → ℝ)
  (hf : ∀ x > 0, f x = x ^ 2)
  : f t = t ^ 2 :=
by
  apply hf
  -- ⊢ t > 0
  exact ht

-- 3ª demostración
-- ===============

example
  (t : ℝ)
  (ht : t > 0)
  (f : ℝ → ℝ)
  (hf : ∀ x > 0, f x = x ^ 2)
  : f t = t ^ 2 :=
by
  exact hf t ht

-- 4ª demostración
-- ===============

example
  (t : ℝ)
  (ht : t > 0)
  (f : ℝ → ℝ)
  (hf : ∀ x > 0, f x = x ^ 2)
  : f t = t ^ 2 :=
by
```

```
  rw [hf]
  -- ⊢ t > 0
  exact ht

-- 5ª demostración
-- ===============

example
  (t : ℝ)
  (ht : t > 0)
  (f : ℝ → ℝ)
  (hf : ∀ x > 0, f x = x ^ 2)
  : f t = t ^ 2 :=
by
  simp_all

-- 6ª demostración
-- ===============

example
  (t : ℝ)
  (ht : t > 0)
  (f : ℝ → ℝ)
  (hf : ∀ x > 0, f x = x ^ 2)
  : f t = t ^ 2 :=
by
  aesop

-- 7ª demostración
-- ===============

example
  (t : ℝ)
  (ht : t > 0)
  (f : ℝ → ℝ)
  (hf : ∀ x > 0, f x = x ^ 2)
  : f t = t ^ 2 :=
by
  solve_by_elim
```

1.9. El desafío táctico definitivo

El desafío final

Aquí tienes la "Hoja de trucos de los cuantificadores":

Empieza con:	∀	∃
Objetivo	`intro`	`use`
Hipótesis	`specialize`	`choose`

Teorema 1.9.1. *Sea $f : \mathbb{R} \to \mathbb{R}$ una función tal que existe un $a \in \mathbb{R}$ con $f(a) = 3$, y para todo $x > 0$, se cumple $f(x+1) = f(x)+9$. Entonces, existe un $b \in \mathbb{R}$ tal que para todo $y > 0$, se tiene $f(y+1)^2 = (f(y)+f(b)^2)^2$.*

```
-- 1ª demostración
example
  (f : ℝ → ℝ)
  (h1 : ∃ a : ℝ, f a = 3)
  (h2 : ∀ x > 0, f (x + 1) = f x + 9)
  : ∃ b : ℝ, ∀ y > 0, f (y + 1) ^ 2 = (f y + (f b) ^ 2)
 ↪  ^ 2 :=
by
  choose a ha using h1
  -- a : ℝ
  -- ha : f a = 3
  use a
  -- ⊢ ∀ y > 0, f (y + 1) ^ 2 = (f y + f a ^ 2) ^ 2
  intro y hy
  -- y : ℝ
  -- hy : y > 0
  -- ⊢ f (y + 1) ^ 2 = (f y + f a ^ 2) ^ 2
  specialize h2 y hy
```

```
  -- h2 : f (y + 1) = f y + 9
  rewrite [h2]
  -- ⊢ (f y + 9) ^ 2 = (f y + f a ^ 2) ^ 2
  rewrite [ha]
  -- ⊢ (f y + 9) ^ 2 = (f y + 3 ^ 2) ^ 2
  ring_nf

-- 2ª demostración
example
  (f : ℝ → ℝ)
  (h1 : ∃ a : ℝ, f a = 3)
  (h2 : ∀ x > 0, f (x + 1) = f x + 9)
  : ∃ b : ℝ, ∀ y > 0, f (y + 1) ^ 2 = (f y + (f b) ^ 2)
 ↪  ^ 2 :=
by
  obtain ⟨a, ha⟩ := h1
  -- a : ℝ
  -- ha : f a = 3
  use a
  -- ⊢ ∀ y > 0, f (y + 1) ^ 2 = (f y + f a ^ 2) ^ 2
  intro y hy
  -- y : ℝ
  -- hy : y > 0
  -- ⊢ f (y + 1) ^ 2 = (f y + f a ^ 2) ^ 2
  rw [h2 y hy]
  -- ⊢ (f y + 9) ^ 2 = (f y + f a ^ 2) ^ 2
  rw [ha]
  -- ⊢ (f y + 9) ^ 2 = (f y + 3 ^ 2) ^ 2
  norm_num

-- 3ª demostración
example
  (f : ℝ → ℝ)
  (h1 : ∃ a : ℝ, f a = 3)
  (h2 : ∀ x > 0, f (x + 1) = f x + 9)
  : ∃ b : ℝ, ∀ y > 0, f (y + 1) ^ 2 = (f y + (f b) ^ 2)
 ↪  ^ 2 :=
by
  obtain ⟨a, ha⟩ := h1
  -- a : ℝ
```

```
  -- ha : f a = 3
  use a
  -- ⊢ ∀ y > 0, f (y + 1) ^ 2 = (f y + f a ^ 2) ^ 2
  grind

-- 4ª demostración
example
  (f : ℝ → ℝ)
  (h1 : ∃ a : ℝ, f a = 3)
  (h2 : ∀ x > 0, f (x + 1) = f x + 9)
  : ∃ b : ℝ, ∀ y > 0, f (y + 1) ^ 2 = (f y + (f b) ^ 2)
↪   ^ 2 :=
by
  rcases h1 with ⟨a, ha⟩
  -- a : ℝ
  -- ha : f a = 3
  use a
  -- ⊢ ∀ y > 0, f (y + 1) ^ 2 = (f y + f a ^ 2) ^ 2
  grind
```

Epílogo

¡Victoria! Has completado un argumento matemático genuinamente sofisticado. Usar un demostrador de teoremas es como aprender ajedrez: es mucho más fácil ver el tablero que leer solo la notación. Esta retroalimentación inmediata construye la intuición necesaria para el cálculo riguroso.

2. El cálculo de Newton del número pi

2.1. La revolución matemática de 1666

SIMPLICIO: He oído que Newton tenía una forma muy ingeniosa de calcular π. ¿Podría hablarme de ello?

SÓCRATES: Por supuesto. Todo empieza alrededor de 1665-1666, cuando Newton estaba a punto de cumplir 23 años. ¿Recuerdas algo significativo sobre ese año?

SIMPLICIO: ¿No es ese el *"annus mirabilis"* de Newton, su año de los milagros? Si mal no recuerdo, se vio obligado a abandonar Cambridge debido a un brote de la Gran Peste, y realizó sus descubrimientos más trascendentales (cálculo, óptica, gravitación, etc.) mientras guardaba cuarentena en aislamiento en su casa familiar de Woolsthorpe.

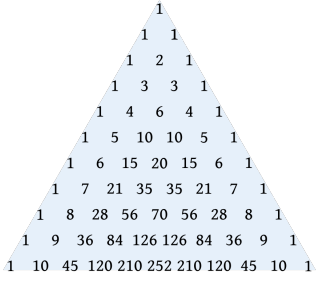
SÓCRATES: Exactamente. Y una de las primeras cosas que descubrió en ese año fue una nueva versión del teorema del binomio. Dime, por favor, ¿qué se puede decir sobre $(1 + x)^n$?

SIMPLICIO: Claro, si desarrollas $(1 + x)^n$, obtienes:

$$\binom{n}{0} + \binom{n}{1}x + \binom{n}{2}x^2 + \cdots + \binom{n}{n}x^n$$

Aquí $\binom{n}{k}$ es el "coeficiente binomial", el número de formas de elegir k objetos de un conjunto de n. Explícitamente, $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Estos son simplemente los números del triángulo de Pascal, y puedes leer fácilmente la fila n-ésima.

Triángulo de Pascal



SÓCRATES: ¡Excelente! ¿Y conoces la notación sigma?

SIMPLICIO: Creo que sí. Podría haber escrito lo mismo como $\sum_{k=0}^{n} \binom{n}{k}x^k$. En general, si tienes una función $f : \mathbb{N} \to \mathbb{R}$ y quieres expresar $f(a)+f(a+1)+\cdots+f(b)$, es decir, la suma de $f(k)$ mientras k varía desde un entero a hasta otro entero b, puedes escribirlo como $\sum_{k=a}^{b} f(k)$.

SÓCRATES: Muy bien. Así que tenemos $(1 + x)^n = \sum_{k=0}^{n} \binom{n}{k}x^k$. Ahora, ¿me permitirías escribir esto como

una suma que llegue hasta el infinito?

$$(1+x)^n = \sum_{k=0}^{\infty} \binom{n}{k} x^k$$

SIMPLICIO: Hmm. Vale, creo que entiendo a qué te refieres: el triángulo de Pascal tiene ceros implícitos en todas partes por fuera, así que $\binom{n}{k}$ es simplemente cero en cuanto $k > n$. Así que lo has escrito como una suma infinita, aunque secretamente termina tras un número finito de términos. Pero, ¿para qué sirve extenderla así?

SÓCRATES: Bueno, déjame preguntarte esto: ¿se te ocurre alguna forma de dar sentido a esta fórmula cuando $n = -1$?

SIMPLICIO: ¡¿Cómo?! No puedes usar coeficientes binomiales. ¡¿Cómo vas a elegir 3 cosas de un conjunto de -1?! ¡Eso no tiene sentido!

SÓCRATES: Tienes razón, pero muchos de los grandes descubrimientos en matemáticas ocurren cuando te das cuenta de que puedes "romper las reglas" y seguir un patrón más allá de su límite previsto. Ponte en el lugar de Newton; ¿qué se le ocurriría a un genio como él?

SIMPLICIO: Bueno, tenemos esta otra fórmula para los coeficientes binomiales, no en términos combinatorios, sino con factoriales: $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Eso sigue sin ayudar porque, ¡¿qué demonios se supone que es

$(-1)!?!$ ¡Ah, espera! También podemos escribirlo así:

$$\binom{n}{k} = \frac{n(n-1)(n-2)\dots(n-k+1)}{k!}$$

Y de esta manera, "esquivamos" el problema de lidiar con $(-1)!$ y pasamos directamente a números "normales".

SÓCRATES: ¿Podrías escribir esto usando la notación de productorio?

SIMPLICIO: Claro, es como la notación de sumatorio pero con una Π:

$$\binom{n}{k} = \frac{1}{k!} \prod_{\ell=1}^{k} (n-\ell)$$

SÓCRATES: ¡Vaya! ¿Estás seguro de los límites de ese producto?

SIMPLICIO: ¡Argh! Es tan fácil cometer un error tonto. Después de escribir la fórmula, debería haber comprobado que los valores de inicio y fin fueran correctos; el contador ℓ debería ir de 0 a $k-1$, no de 1 a k. ¿Así mejor?

$$\binom{n}{k} = \frac{1}{k!} \prod_{\ell=0}^{k-1} (n-\ell)$$

SÓCRATES: Perfecto. Continúa.

SIMPLICIO: Vale, si aceptamos seguir este patrón, obtenemos:

- $\binom{-1}{0} = 1$, lo cual tiene sentido porque cualquier fila del triángulo de Pascal empieza por $1 = \binom{c}{0}$.
- $\binom{-1}{1} = (-1)/1! = -1$, que también tiene sentido porque el siguiente término en la "fila c" es siempre $\binom{c}{1} = c$.
- $\binom{-1}{2} = (-1)(-2)/2! = +1$.
- $\binom{-1}{3} = (-1)(-2)(-3)/3! = -1$.

Ya veo el patrón: simplemente alterna entre $+1$ y -1, así que la serie queda:

$$(1+x)^{-1} = 1 - x + x^2 - x^3 + x^4 - x^5 + \ldots$$

SÓCRATES: ¡Muy bien! Pero, ¿tiene esta fórmula algún sentido real? ¿Cómo podrías ponerla a prueba?

SIMPLICIO: Vale, si se supone que esa serie "representa" a $1/(1+x)$, entonces si multiplico todo por $(1+x)$, debería obtener simplemente 1. Vamos a probar:

$$(1+x)(1 - x + x^2 - x^3 + x^4 - x^5 + \ldots) = ?$$

Primero multiplicaré todo por 1, luego por x, y los sumaré.

$$(1 - x + x^2 - x^3 + x^4 - x^5 + \ldots) + (x - x^2 + x^3 - x^4 + x^5 \ldots)$$

Si reordeno los términos, todo se cancela y solo queda el 1 inicial. ¡Genial!

SÓCRATES: Interesante. ¿Y se te "permite" reordenar los términos así?

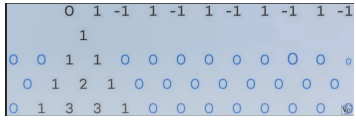
SIMPLICIO: Bueno... ¿por qué no?

SÓCRATES: Olvídalo por ahora; pareces satisfecho con que tenga sentido decir que la serie $1 - x + x^2 - x^3 + x^4 - x^5 + \ldots$ "converge" (sea lo que sea que eso signifique) a $(1 + x)^{-1}$.

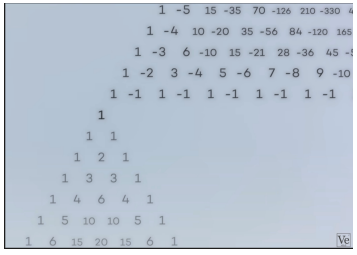
SIMPLICIO: Pensándolo bien, ya sabía esto; ¡es solo la serie geométrica! Sé que $1 + \lambda + \lambda^2 + \lambda^3 + \ldots$ suma $1/(1-\lambda)$, y nuestra serie simplemente reemplaza λ por $-x$.

SÓCRATES: Sí, muy bien. ¿Y dónde podría "encajar" esto en el triángulo de Pascal?

SIMPLICIO: ¡Cielos! ¿Acabamos de descubrir una extensión del triángulo que va hacia "arriba"?



SÓCRATES: En efecto, y de hecho podemos continuar este patrón para $n = -2, -3, -4$, y así sucesivamente. Te dejaré que lo deduzcas tú mismo, ¡pero en realidad obtenemos todo otro Triángulo de Pascal (con algunos signos negativos) por encima del estándar!



¿Ves cómo sigue cumpliendo la regla habitual de que los dos números de arriba sumen el valor que tienen justo debajo? Pero probemos algo aún más exótico. ¿Puedes hacer que el Teorema del Binomio funcione para $n = 1/2$?

SIMPLICIO: ¡Hala! ¿$n = 1/2$? Eso es... ¡forzar mucho las cosas! Pero déjame intentar usar la misma fórmula. Así que $\binom{1/2}{k} = \frac{1}{k!}\prod_{\ell=0}^{k-1}(1/2 - \ell)$. Voy a calcular los primeros términos:

- $\binom{1/2}{0} = 1$ (como siempre).
- $\binom{1/2}{1} = (1/2)/1! = 1/2$ (de nuevo, coincide con el patrón que ya conocíamos).
- $\binom{1/2}{2} = (1/2)(-1/2)/2! = -1/8$.
- $\binom{1/2}{3} = (1/2)(-1/2)(-3/2)/3! = 1/16$.
- $\binom{1/2}{4} = \frac{(1/2)(-1/2)(-3/2)(-5/2)}{4!} = -5/128$.

Así que $(1+x)^{1/2} = 1 + \frac{1}{2}x - \frac{1}{8}x^2 + \frac{1}{16}x^3 - \frac{5}{128}x^4 + \ldots$ Pero espera... ¡¿se supone que esto es $\sqrt{1+x}$?!

SÓCRATES: De nuevo, entra en la mente de Newton: ¿cómo podría él justificar si esta fórmula tiene algún sentido?

SIMPLICIO: ¡Ah, ya veo! Si elevamos la fórmula al cuadrado y lo multiplicamos todo, supongo que deberíamos obtener $1 + x$; eso justificaría llamar a la serie $\sqrt{1+x}$.

SÓCRATES: ¡Adelante!

SIMPLICIO: Vale, quiero calcular:

$$\left(1 + \frac{1}{2}x - \frac{1}{8}x^2 + \frac{1}{16}x^3 - \frac{5}{128}x^4 + \ldots\right)^2$$

Eso significa elevar cada término al cuadrado, y también añadir el doble de cada producto de pares distintos de términos.

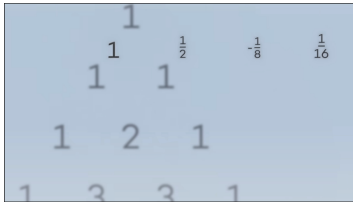
SÓCRATES: Correcto. ¿Se te ocurre una buena forma de llevar la cuenta de todo?

SIMPLICIO: ¡Oh, ya sé! Vamos a agrupar las cosas por la potencia de x involucrada. La primera es fácil: $x^0 = 1$, que solo se obtiene elevando al cuadrado el primer término. Así que ese coeficiente es 1.
Para el coeficiente de x^1, no puedo elevar al cuadrado nada que contenga x, así que solo puedo multiplicar el término en x por el término constante y, por supuesto, duplicarlo. Eso es simplemente $2 \times 1 \times \frac{1}{2}x = x$. Así que el coeficiente de x es 1.
Para x^2, obtengo dos contribuciones: de constante por cuadrático, $2 \times 1 \times (-\frac{1}{8}x^2) = -\frac{1}{4}x^2$, y también del cuadrado del término lineal: $(\frac{1}{2}x)^2 = \frac{1}{4}x^2$. Así que el coefi-

ciente total es $-\frac{1}{4} + \frac{1}{4} = 0$.
Probemos unos pocos más. Para obtener x^3, necesito:

- $2 \times 1 \times \frac{1}{16}x^3 = \frac{1}{8}x^3$ (constante por el término x^3)
- $2 \times \frac{1}{2}x \times (-\frac{1}{8}x^2) = -\frac{1}{8}x^3$ (el término x por el término x^2)

Así que el coeficiente total de x^3 es $\frac{1}{8} - \frac{1}{8} = 0$.
¡Esto es increíble! Parece que todos los términos de orden superior se cancelan perfectamente. Apuesto a que eso seguirá pasando y el cuadrado resultará ser exactamente $1 + x$; ¡la fórmula realmente funciona!
Pero espera, ¡¿ahora tenemos otra fila entera en el triángulo de Pascal, entre las filas 0 y 1?!



SÓCRATES: Hermoso, ¿verdad?

SIMPLICIO: Espera, esto es mucho más sencillo de lo que parece. ¿No es esto simplemente lo mismo que el desarrollo de Taylor alrededor de $x = 0$ de la función $f(x) = \sqrt{1 + x}$? Ya sé cómo hacer eso por mis clases de Cálculo.

SÓCRATES: Sí, muy bien; pero Brook Taylor (el de las series de Taylor) no demostró su teorema general

hasta 1715, unas décadas después del cálculo de π de Newton.

SÓCRATES: Ahora, supón que quisieras calcular algo como $\sqrt{3}$. ¿Se te ocurre alguna forma de hacerlo usando esta fórmula?

SIMPLICIO: Hmm, la función es $\sqrt{1+x}$, así que supongo que quiero poner $x = 2$. Entonces obtengo:

$$\sqrt{1+2} = 1 + \frac{1}{2}2 - \frac{1}{8}2^2 + \frac{1}{16}2^3 - \frac{5}{128}2^4 + \ldots$$

Sumar estos cinco términos da $11/8 = 1,375$, que no está muy cerca de $\sqrt{3} \approx 1,73$. Además, los términos individuales no son tan pequeños; por ejemplo, el último es $\frac{5}{128}2^4 = 5/8 = 0,625$.

SÓCRATES: Bueno, claro, si eliges una x grande, como $x > 1$, entonces las potencias de x son cada vez más grandes (¡y exponencialmente!). ¿Se te ocurre alguna otra cosa que pudieras hacer?

SIMPLICIO: ¡Ah, ya veo! Sé que 3 está cerca de 4, que es un cuadrado perfecto. ¿Y si escribimos:

$$\sqrt{3} = \sqrt{4-1} = \sqrt{4(1-1/4)} = 2\sqrt{1-1/4}$$

Así que ahora, si aplico nuestra fórmula con $x = -1/4$ (¡que es menor que uno!), supongo que obtendré:

$$\sqrt{3} \approx 2\left(1 + \frac{1}{2}(-1/4) - \frac{1}{8}(-1/4)^2 + \frac{1}{16}(-1/4)^3 - \frac{5}{128}(-1/4\right.$$

Tomando solo estos cinco términos, la fracción de la derecha sale $28379/16384 \approx 1,73212$, que es impresionantemente cercana a $\sqrt{3} \approx 1,7320508$. ¡Hemos obtenido 3 decimales de precisión, nada mal!

SÓCRATES: ¡Genial! Ahora ves el poder del teorema del binomio de Newton. Vale, volvamos entonces a tu pregunta original sobre la estimación de π de Newton.

SIMPLICIO: Hmmm, π es la relación entre la circunferencia y el diámetro de un círculo. Pero, ¿de dónde voy a sacar una longitud?

SÓCRATES: ¡Ah! Pero, ¿qué aprendimos de Arquímedes?

SIMPLICIO: ¡Oh! Que π también es un *área*, no solo una longitud. Es el área de un círculo unidad πr^2 donde $r = 1$.

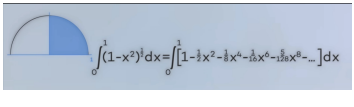
SÓCRATES: Precioso. ¿Y podrías encontrar el área de un círculo escondida en alguna parte?

SIMPLICIO: ¡Creo que lo veo! Gracias a Descartes y a las coordenadas "cartesianas", podemos expresar el círculo como la gráfica de $x^2+y^2 = 1$, o para convertirlo en una función, $y = \sqrt{1-x^2}$. Así que solo tenemos que reemplazar x en nuestra serie por $-x^2$.

$$\sqrt{1-x^2} = 1+\frac{1}{2}(-x^2)-\frac{1}{8}(-x^2)^2+\frac{1}{16}(-x^2)^3-\frac{5}{128}(-x^2)^4+\ldots$$

$$= 1 - \frac{x^2}{2} - \frac{x^4}{8} - \frac{x^6}{16} - \frac{5x^8}{128} - \ldots$$

Y el área bajo la curva $y = \sqrt{1-x^2}$ desde $x = 0$ hasta $x = 1$ es un cuarto de círculo.



SÓCRATES: ¡Por suerte, Newton acababa de inventar el cálculo! Entonces, ¿de qué otra forma podría calcular el área bajo esta curva?

SIMPLICIO: ¡Con una integral! Así que:

$$\frac{\pi}{4} = \int_0^1 \sqrt{1-x^2}\,dx = \int_0^1 \left(1 - \frac{x^2}{2} - \frac{x^4}{8} - \frac{x^6}{16} - \frac{5x^8}{128} - \ldots\right) d$$

Simplemente integraré término a término...

SÓCRATES: ¡Epa, espera un momento! ¿Por qué se te permite...? Sabes qué, da igual, lo siento. Adelante.

SIMPLICIO: Qué raro eres. En fin. Integrando término a término obtengo:

$$\frac{\pi}{4} = \left[x - \frac{x^3}{6} - \frac{x^5}{40} - \frac{x^7}{112} - \frac{5x^9}{1152} - \ldots\right]_0^1$$

$$= 1 - \frac{1}{6} - \frac{1}{40} - \frac{1}{112} - \frac{5}{1152} - \ldots$$

¡Vaya! ¡Así que Newton obtuvo una serie infinita para π! Si evalúo solo estos cinco términos y multiplico por 4, obtengo la fracción $32057/10080 \approx 3,180$. ¡No está mal!

SÓCRATES: No está mal, en efecto. Sabes, Simplicio, muchos artículos de matemáticas tienen prácticamente cero ideas nuevas; simplemente hacen algo que nadie se molestó en hacer antes en un contexto ligeramente distinto. Un artículo de matemáticas realmente bueno puede tener una o dos ideas genuinamente nuevas. ¡Newton ya va por la idea nueva número cinco, y aún no ha terminado!

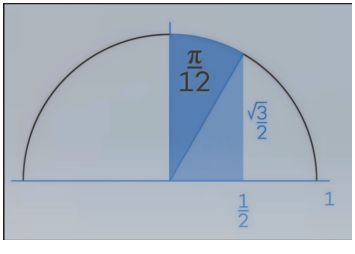
SIMPLICIO: Vale, ¿y cuál es la idea nueva número seis?

SÓCRATES: Bueno, ¿recuerdas que integraste hasta $x = 1$? En tu serie,

$$\frac{\pi}{4} = \left[x - \frac{x^3}{6} - \frac{x^5}{40} - \frac{x^7}{112} - \frac{5x^9}{1152} - \dots \right]_0^1$$

tienes todas estas potencias altas de x, pero se están "desperdiciando" porque estás poniendo $x = 1$. ¿Y si en su lugar solo integraras hasta, por ejemplo, $x = 1/2$?

SIMPLICIO: ¡Ah, qué guay! Entonces la serie convergerá mucho más rápido. Pero espera, eso cambia la geometría. En lugar de un cuarto de círculo, ahora tenemos... un sector de 30 grados, que tiene un área de $\pi/12$, más un triángulo 30-60-90 (¡ah, por eso sugeriste $x = 1/2$!) con área $\frac{1}{2} \times \frac{1}{2} \times \frac{\sqrt{3}}{2}$.



¡Menos mal que ya sabemos cómo estimar $\sqrt{3}$ rápidamente con gran precisión! (Esa es la contrapartida: podríamos elegir una x aún más pequeña para una convergencia más rápida, pero entonces tendríamos que

lidiar con evaluaciones geométricas cada vez más complicadas; así que $x = 1/2$ es el "punto ideal"). Así que ahora:

$$\frac{\pi}{12} + \frac{\sqrt{3}}{8} = \left[x - \frac{x^3}{6} - \frac{x^5}{40} - \frac{x^7}{112} - \frac{5x^9}{1152} - \dots \right]_0^{1/2}$$

$$= \frac{1}{2} - \frac{1}{6}\left(\frac{1}{2}\right)^3 - \frac{1}{40}\left(\frac{1}{2}\right)^5 - \frac{1}{112}\left(\frac{1}{2}\right)^7 - \frac{5}{1152}\left(\frac{1}{2}\right)^9 - \dots$$

Evaluando de nuevo solo estos cinco términos obtenemos la fracción 9874097/20643840. Y ahora, aislando π obtenemos la estimación:

$$\pi \approx 12 \times \left(9874097/20643840 - \frac{\sqrt{3}}{8} \right) \approx 3,14161,$$

que solo difiere en dos partes por cada 100.000 de la estimación real $\pi \approx 3,14159$. ¡Y todo eso con solo cinco términos, increíble!

SÓCRATES: Sí, Newton fue realmente impresionante. Aquí tienes un buen vídeo de YouTube de Veritasium que analiza toda esta saga: https://youtu.be/gMlf1ELvRzc

De hecho, una serie para π similar a esta fue descubierta dos siglos antes por el matemático indio Madhava de Sangamagrama. Y harían falta dos siglos más hasta que los matemáticos descubrieran cómo justificar rigurosamente el trabajo de Newton. Para hacerlo, tuvieron que averiguar:

- ¿Qué significa que una sucesión de números reales $a_0, a_1, a_2, \dots$ converja?

- ¿Qué significa que una serie (es decir, una sucesión de sumas parciales) $a_0+a_1+a_2+\ldots$ converja, y podríamos sumar estos números en cualquier orden que quisiéramos?
- ¿Qué significa que una serie que involucra una variable, como una serie de potencias $a_0 + a_1x + a_2x^2 + \ldots$, converja, y si es así, a qué tipo de función converge?
- ¿Cuándo podemos intercambiar límites con integrales, como al integrar término a término?

$$\int (a_0+a_1x+a_2x^2+\ldots)dx \stackrel{?}{=} \int a_0dx+\int a_1xdx+\int a_2x^2d$$

Etcétera, etcétera. ¡Tenemos mucho trabajo por hacer!

SIMPLICIO: Vale, vale; ¡me has convencido! Empecemos con algo de análisis real de verdad, por favor.

2.2. La definición principal

Nuestro primer paso para dar rigor al argumento de Newton es deletrear exactamente qué queremos decir cuando una sucesión a_n converge. Costará un poco de trabajo construir la definición y, lo que es más importante, entender *por qué* parece una definición razonable.

Pero primero: por alguna razón (probablemente la culpa sea de Euler), las matemáticas tienen dos convenciones completamente diferentes para escribir funciones.

Para funciones generales $f : \mathbb{R} \to \mathbb{R}$, escribimos $f(x)$, con paréntesis. Pero cuando trabajamos con sucesiones, a_n, es decir, $a_0, a_1, a_2, \ldots$, cambiamos extrañamente a subíndices. ¿Por qué? Accidente histórico.

Una sucesión no es más que una función cuyo "dominio" (es decir, el conjunto de entradas a la función) son los números naturales; así que romperemos con la tradición y unificaremos las dos convenciones, escribiendo de ahora en adelante $a : \mathbb{N} \to \mathbb{R}$ para sucesiones de números reales, $a(0), a(1), a(2), \ldots$.

Ahora bien, la definición que los matemáticos acabaron creando para lo que significa que una sucesión converja era tan intrincada (al menos a primera vista) ¡que tuvo que ser inventada dos veces!

La formulación final cristalizó a través del trabajo de Karl Weierstrass en la década de 1860, quien transformó el análisis de un arte intuitivo en una ciencia rigurosa. Sin embargo, las semillas de esta idea aparecieron mucho antes en el trabajo de Bernard Bolzano. En las décadas de 1810 y 1820, Bolzano estaba desarrollando ideas asombrosamente modernas sobre la continuidad y los límites, pero estaba demasiado adelantado a su tiempo para que la comunidad matemática aceptara estos conceptos abstractos. Solo en la época de Weierstrass —medio siglo después— calaron estas ideas.

Sin más preámbulos, aquí está:

Definición 2.2.1 (Convergencia de sucesiones)**.** Dada una sucesión $a : \mathbb{N} \to \mathbb{R}$ y un número real $L : \mathbb{R}$, escri-

bimos $\lim a = L$ y decimos que la sucesión a **converge** a L, si: para cada $\varepsilon > 0$, existe un $N \in \mathbb{N}$ tal que, para todo $n \geq N$, tenemos $|a(n) - L| < \varepsilon$.

Esta definición probablemente no sea la primera, ni la segunda, ni la décima cosa que se te habría ocurrido. Pero con el tiempo, espero que llegues a ver que encarna una hermosa negociación entre precisión y esfuerzo.

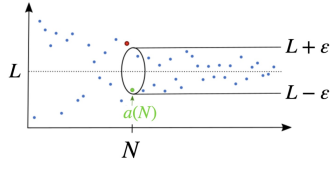
Me gusta pensar en ello como una conversación entre un ingeniero y un maquinista. El ingeniero llega con las especificaciones: "Vamos a fabricar esta pieza, y necesito que su longitud sea de 1 pie, con una tolerancia de error de 1/100 de pulgada".

El maquinista responde: "Claro, puedo hacerlo, pero tendré que hacer funcionar mi equipo especial durante al menos 10 horas para garantizar esa tolerancia". El ingeniero responde: "Lo siento, me equivoqué, ¿podemos cambiar la tolerancia a 1/1000 de pulgada?". El maquinista responde: "Uf, sí, podemos hacerlo, pero te costará. Necesitaré al menos 40 horas de operación, pero después de eso, te lo garantizo".

Mientras esta conversación pueda continuar independientemente de *cualquier* tolerancia $\varepsilon > 0$ que requiera el ingeniero, siendo el maquinista siempre capaz de responder con un número mínimo finito de horas N, tras el cual se alcanzará la tolerancia, podemos decir que el equipo **converge**.

Leamos ahora la definición de Weierstrass (¿o es la de

Bolzano?) otra vez. Tenemos algún proceso que en el tiempo n devuelve una lectura $a(n)$ (piensa: longitud de la pieza). Nuestro objetivo final es conseguir la longitud L. Si para cualquier tolerancia $\varepsilon > 0$, por pequeña que sea, siempre existe algún tiempo mínimo N, de modo que, para cualquier tiempo futuro $n \geq N$, se garantiza que estaremos dentro de esa tolerancia, $|a(n) - L| < \varepsilon$, esa es exactamente la condición bajo la cual diremos que la sucesión $a(n)$ **converge** a L.



Lo que hace que esta definición sea tan poderosa es su universalidad. El maquinista está esencialmente prometiendo: "Dame cualquier requisito de tolerancia, por estricto que sea, y puedo cumplirlo, aunque necesite más recursos (un N mayor) para especificaciones más ajustadas".

Fíjate en algo más sobre la definición: no hace mención a que algo suceda "eventualmente", o "en el infinito" ni ninguna otra palabra ambigua y escurridiza. Hemos cambiado la ambigüedad de hablar del infinito por la precisión de los cuantificadores existenciales y universales. No más imprecisiones sobre lo que sucede "cuando n se hace grande"; en su lugar, tenemos un desafío concreto: dada cualquier tolerancia ε, ¿puedes encontrar un umbral específico N? Esa idea fue el

avance clave que permitió que el Cálculo entrara en el reino de las matemáticas rigurosas.

En Lean, la definición se escribe así:

```
def LimSuc (a : ℕ → ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, |a n - L| < ε
```

Esta sintaxis debería resultarte familiar por la táctica `have` que ya conoces. El símbolo especial `def` (en lugar de `have`) significa que estamos a punto de definir algo, y `LimSuc` es su nombre (por *límite de sucesión*, por supuesto; pero podríamos haberlo llamado como quisiéramos). Luego nuestras suposiciones son una sucesión `a : N →R` y algún número real `L ∈R`. Después de los dos puntos `:` va nuestra salida, que en este caso es `Prop`, es decir, una proposición (sentencia) que puede ser verdadera o falsa. Así que `LimSuc` es en realidad una función que toma una sucesión y un hipotético valor límite, y devuelve verdadero o falso dependiendo de si se cumple la condición. Luego viene un signo de dos puntos e igual `:=`, tras el cual se especifica la condición exacta que se va a probar. Y la condición es lo que ya dijimos: para todo epsilon, ... La gran diferencia es que puedes escribir `have` dentro de una demostración, pero no puedes escribir `def` dentro de una; `def` está reservado para crear definiciones globales a las que se puede hacer referencia para siempre una vez introducidas.

Puede que te resulte útil una nueva táctica llamada `change`. Te permite reemplazar un objetivo (o hipóte-

sis) por algo que es definicionalmente igual a él. En nuestro ejemplo, verás el objetivo como `LimSuc a L`. ¿Qué se supone que debes hacer con eso? Bueno, si recuerdas cómo se define `LimSuc`, entonces puedes reemplazar el objetivo con la definición, escribiendo:

```
change ∀ ε > 0 , ∃ N : ℕ, ∀ n ≥ N , |a n - L| < ε
```

Lean cambiará entonces el objetivo por su definición. Recuerda que ε, N y n son todas variables mudas aquí, así que puedes divertirte un poco:

```
change ∀ Alicia > 0 , ∃ Bob : ℕ, ∀ Blas ≥ Bob , |a Blas
↪  - L| < Alicia
```

Y una última táctica que también podrías encontrar útil es `norm_num` (para normalizar valores numéricos); evalúa expresiones numéricas y demuestra igualdades/desig que involucren números concretos. Por ejemplo, si te quedas atascado con un $|0|$ en algún momento y quieres convertirlo en un simple 0, intenta llamar a `norm_num`.

```
--
↪ ----------------------------------------------------------
-- Demostrar que si a es una sucesión tal que para todo
-- n, a(n) = L, entonces a converge a L.
--
↪ ----------------------------------------------------------
```

```
-- Demostración en lenguaje natural
-- ================================

-- Tenemos que demostrar que para cada ε ∈ ℝ tal que ε >
 ↪  0, existe un
-- N ∈ ℕ, tal que (∀n ∈ ℕ)[n ≥ N → |a(n) - L| < ε].
 ↪  Basta tomar N como
-- 0, ya que para todo n ≥ N se tiene
--    |a(n) - L| = |L - L|
--               = |0|
--               = 0
--               < ε

-- Demostraciones con Lean4
-- ========================

variable (a : ℕ → ℝ)
variable (L : ℝ)

-- 1ª demostración
-- ===============

example
  (h : ∀ n, a n = L)
  : LimSuc a L :=
by
  change ∀ ε > 0 , ∃ N : ℕ, ∀ n ≥ N , |a n - L| < ε
  -- ⊢ ∀ ε > 0, ∃ N, ∀ n ≥ N, |a n - L| < ε
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |a n - L| < ε
  use 0
  -- ⊢ ∀ n ≥ 0, |a n - L| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ 0
  -- ⊢ |a n - L| < ε
  specialize h n
```

```
  -- h : a n = L
  rewrite [h]
  -- ⊢ |L - L| < ε
  ring_nf
  -- ⊢ |0| < ε
  norm_num
  -- ⊢ 0 < ε
  exact hε

-- 2ª demostración
-- ===============

example
  (h : ∀ n, a n = L)
  : LimSuc a L :=
by
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ (n : ℕ), n ≥ N → |a n - L| < ε
  use 0
  -- ⊢ ∀ (n : ℕ), n ≥ 0 → |a n - L| < ε
  intros n hn
  -- n : ℕ
  -- hn : n ≥ 0
  -- ⊢ |a n - L| < ε
  show |a n - L| < ε
  calc |a n - L| = |L - L| := by rw [h n]
               _ = |0|     := by {congr ; exact
↪  sub_self L}
               _ = 0       := abs_zero
               _ < ε       := hε

-- 3ª demostración
-- ===============

example
  (h : ∀ n, a n = L)
  : LimSuc a L :=
by
```

```
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ (n : ℕ), n ≥ N → |a n - L| < ε
  use 0
  -- ⊢ ∀ (n : ℕ), n ≥ 0 → |a n - L| < ε
  intros n hn
  -- n : ℕ
  -- hn : n ≥ 0
  -- ⊢ |a n - L| < ε
  show |a n - L| < ε
  calc |a n - L| = 0 := by simp [h n]
               _ < ε := hε

-- 4ª demostración
-- ===============

example
  (h : ∀ n, a n = L)
  : LimSuc a L :=
by
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ (n : ℕ), n ≥ N → |a n - L| < ε
  aesop

-- 5ª demostración
-- ===============

example
  (h : ∀ n, a n = L)
  : LimSuc a L :=
by
  intros ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ (n : ℕ), n ≥ N → |a n - L| < ε
  aesop
```

```
-- 6ª demostración
-- ===============

example
  (h : ∀ n, a n = L)
  : LimSuc a L :=
fun ε hε ↦ by aesop

-- Lemas usados
-- ============

#check (sub_self a : a - a = 0)
```

¡Acabas de completar tu primera demostración rigurosa de un límite! Reflexionemos sobre lo que has logrado y las ideas clave de este ejemplo fundacional.

Lo que acabas de demostrar: Has demostrado que si una sucesión siempre devuelve el mismo valor L, entonces converge a L. La respuesta del maquinista a cualquier demanda de tolerancia $\varepsilon > 0$ es bellamente simple: "¡Puedo cumplir esa especificación inmediatamente con cualquier longitud de producción N, porque ya estoy produciendo exactamente lo que quieres!".

Ideas clave de esta demostración:

1. **La táctica `change`:** Aprendiste a desplegar una definición para ver qué es lo que realmente estás intentando demostrar. `LimSuc a L` se convirtió en la condición concreta epsilon–N.

2. **La estructura lógica:** La demostración siguió

el flujo natural de la definición:

- `intro ε hε` manejó el "para todo $\varepsilon > 0$".
- `use 1` proporcionó el testigo N (¡cualquier número sirve!).
- `intro n hn` manejó el "para todo $n \geq N$".
- Luego la manipulación algebraica mostró que $|a(n) - L| = |L - L| = |0|$.
- Después, la normalización numérica dio que $|0| = 0$, y `hε` finalmente demostró que $0 < \varepsilon$.

La hermosa simplicidad: Este es el escenario soñado por el maquinista: no importa cuán exigentes sean los requisitos de tolerancia del ingeniero, la fábrica de constantes puede satisfacerlos instantáneamente. ¡No hay compromiso entre precisión y esfuerzo porque el resultado ya es perfecto!

Estás construyendo la base de todo el cálculo. Cada límite, derivada e integral descansa en última instancia en argumentos como este.

Alejémonos un poco de la demostración formal de Lean y entendamos qué acabamos de demostrar en lenguaje natural.

Teorema 2.2.2. *Si una sucesión tiene el mismo valor para cada término, entonces converge a ese valor constante.*

Demostración: Supongamos que tenemos una sucesión $a(n)$ donde $a(n) = L$ para todo n, y queremos demostrar que esta sucesión converge a L.

Por definición, necesitamos demostrar que para cualquier tolerancia $\varepsilon > 0$, podemos encontrar un punto N tal que para todo $n \geq N$, tenemos $|a(n) - L| < \varepsilon$.

Esto es casi trivialmente simple: como $a(n) = L$ para cada n, tenemos:

$$|a(n) - L| = |L - L| = |0| = 0$$

Como $0 < \varepsilon$ para cualquier ε positivo, podemos elegir cualquier N que queramos (elegimos $N = 1$ en la demostración, pero $N = 0$ o $N = 1000$ funcionarían igual de bien).

Por lo tanto, para cualquier $n \geq N$, tenemos $|a(n)-L| = 0 < \varepsilon$, lo que demuestra la convergencia. □

3. Convergencia de sucesiones

3.1. Más sobre sucesiones

En el Tema 2, aprendimos la definición de que una sucesión $a : \mathbb{N} \to \mathbb{R}$ converge a un límite L: para cualquier tolerancia $\varepsilon > 0$, existe un momento N, de modo que, para cualquier punto posterior a ese momento, $n \geq N$, estamos dentro de la tolerancia, $|a_n - L| < \varepsilon$.

También aprendimos cómo hacer algo completamente trivial con ella, concretamente, demostrar que la sucesión constante converge, con esa constante como su límite. Vamos a subir un peldaño, ¿de acuerdo?

3.2. Nivel 1: La propiedad arquimediana

La llamada propiedad arquimediana (que creo que se debe originalmente a Eudoxo, y aparece ya en los *Elementos* de Euclides, Libro V) es una propiedad fundamental de los números reales que captura la noción intuitiva de que no existen números reales positivos "infinitamente grandes" o "infinitamente pequeños".

Más precisamente, establece que no importa cuán pequeño sea $\varepsilon > 0$, siempre existe un número natural N de modo que $1/N$ es aún más pequeño que ε (y, por supuesto, positivo). Equivalentemente, podemos enunciarla como: para cualquier número real positivo ε, existe un número natural N tal que $1/\varepsilon < N$.

¿Por qué importa esto? La propiedad arquimediana es una de las propiedades más fundamentales que distinguen a los números reales de otros sistemas numéricos. Sin ella, podríamos tener números positivos "infinitamente grandes" o "infinitamente pequeños", lo que rompería la mayor parte del cálculo y el análisis.

Nuestro objetivo será demostrar lo siguiente:

Teorema 3.2.1 (PropArq)**.** *Para cualquier $\varepsilon \in \mathbb{R}$ con $0 < \varepsilon$, existe $N \in \mathbb{N}$ tal que $1/\varepsilon < N$.*

Esto es matemáticamente "obvio" para la mayoría de la gente: si tienes un número positivo ε, por muy pequeño que sea, siempre puedes encontrar un número natural lo suficientemente grande para que $1/\varepsilon$ sea menor que él. Pero, ¿cómo se formaliza esto realmente en Lean?

3.2.1. La estrategia de demostración en lenguaje natural

Primero, pensemos en esto en lenguaje natural. La clave es que necesitamos proporcionar un número natural N específico que funcione.

Una opción natural sería utilizar algo relacionado con la función techo (*ceiling*). La función techo $x \mapsto \lceil x \rceil$ redondea cualquier número real al entero más cercano por arriba. Sin embargo, hay un problema sutil aquí: la función techo estándar devuelve valores en los enteros $\mathbb{Z}$, pero necesitamos valores en $\mathbb{N}$ (los números naturales). ¡¡No son lo mismo!!

Afortunadamente, Lean proporciona la "función techo de números naturales", escrita como $x \mapsto \lceil x \rceil_+$, que toma cualquier número real y devuelve un número natural. (Puedes escribir estos símbolos usando `\lceil`, `\rceil` y `\_+`). Para entradas negativas, esta función devuelve 0. Por ejemplo, $\lceil -3{,}14 \rceil_+ = 0$ y $\lceil 3{,}14 \rceil_+ = 4$. Ahora nuestra estrategia queda clara:

- **Elección de N:** Usar $N = \lceil 1/\varepsilon \rceil_+ + 1$
- **Por qué funciona:** Tenemos la "desigualdad clave": $1/\varepsilon \leq \lceil 1/\varepsilon \rceil_+$, que se cumple por la definición de la función techo.
- **Obtener la desigualdad estricta:** Sumar 1 nos da $1/\varepsilon < \lceil 1/\varepsilon \rceil_+ + 1$

3.2.2. Los desafíos de la implementación en Lean

En Lean, los dos primeros pasos de nuestra demostración en lenguaje natural funcionan bien, pero luego

nos encontramos con el problema de la **coerción de tipos** (”casting” entre diferentes tipos de números). Discutiremos esto con más detalle más adelante, pero de nuevo tiene que ver con el hecho de que $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ y $\mathbb{R}$ son todos tipos de cosas diferentes, y necesitamos poder mover los números hacia arriba en la jerarquía de “sofisticación”, siendo los naturales los objetos más simples y los reales los más complicados.

Por ejemplo, fíjate en que cuando escribamos nuestra sentencia `have` para establecer la desigualdad clave:

```
have fact : 1 / ε ≤ ⌈1 / ε⌉₊ := by Demostracion
```

Lean la registrará como:

```
fact : 1 / ε ≤ ↑ ⌈1 / ε⌉₊
```

Observa la misteriosa flecha hacia arriba ↑. Esta representa una función de coerción de los números naturales a los reales:

$$\uparrow : \mathbb{N} \to \mathbb{R}$$

Esto se debe a que $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ y $\mathbb{R}$ son todos tipos **diferentes** en el sistema de tipos de Lean. Aunque pensemos que los naturales están “contenidos” en los reales, formalmente son tipos distintos, y Lean necesita funciones de coerción explícitas para convertir entre ellos.

Piénsalo de esta manera: el número natural $3 \in \mathbb{N}$, la fracción $3/1 \in \mathbb{Q}$ y el número real $3{,}000 \in \mathbb{R}$ son objetos diferentes que simplemente representan el mismo valor matemático.

La táctica `push_cast` ayuda a gestionar estas coerciones, de forma similar a `ring_nf` pero para conversiones en lugar de operaciones de anillo.

3.2.3. Nuevas herramientas que necesitarás

- $\lceil \cdot \rceil_+$: La función techo de números naturales.
- `push_cast`: Táctica que maneja las coerciones entre tipos numéricos.
- `bound`: Resuelve muchas desigualdades rutinarias una vez que los tipos están correctamente alineados.

3.2.4. Demostraciones en Lean

```
import Mathlib.Data.Real.Basic
import Mathlib.Tactic

variable {ε : ℝ}
variable (h : 0 < ε)

-- Demostración en lenguaje natural
-- ================================
```

```
-- Sea N = ⌈1/ε⌉₊ + 1. Entonces,
--    1/ε ≤ ⌈1/ε⌉₊
--        < ⌈1/ε⌉₊ + 1
--        = N
-- Por lo tanto, 1/ε < N.

-- 1ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  have fact : 1 / ε ≤ ⌈1 / ε⌉₊ := by bound
  -- fact : 1 / ε ≤ ↑⌈1 / ε⌉₊
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  bound

-- 2ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  have fact : 1 / ε ≤ ⌈1 / ε⌉₊ := Nat.le_ceil (1 / ε)
  -- fact : 1 / ε ≤ ↑⌈1 / ε⌉₊
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  apply lt_of_le_of_lt fact
  -- ⊢ ↑⌈1 / ε⌉₊ < ↑⌈1 / ε⌉₊ + 1
  exact lt_add_one _

-- 3ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
```

```
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  calc
    1 / ε ≤ ↑⌈1 / ε⌉₊     := Nat.le_ceil (1 / ε)
    _     < ↑⌈1 / ε⌉₊ + 1 := lt_add_one _

-- 4ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  push_cast
  -- ⊢ 1 / ε < ↑⌈1 / ε⌉₊ + 1
  exact lt_of_le_of_lt (Nat.le_ceil (1 / ε))
 ↪  (lt_add_one _)

-- 5ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
exists_nat_gt (1 / ε)

-- 6ª demostración
-- ===============

example : ∃ (N : ℕ), 1 / ε < N :=
by
  -- ⊢ ∃ N, 1 / ε < ↑N
  use ⌈1 / ε⌉₊ + 1
  -- ⊢ 1 / ε < ↑(⌈1 / ε⌉₊ + 1)
  calc
    1 / ε ≤ (⌈1 / ε⌉₊ : ℝ)     := Nat.le_ceil (1 / ε)
    _     < (⌈1 / ε⌉₊ : ℝ) + 1 := lt_add_one _
```

```
    _      = ↑(⌈1 / ε⌉₊ + 1)     := by norm_num

-- Lemas usados
-- ============

variable (a b c : ℝ)
#check (Nat.le_ceil a : a ≤ ↑⌈a⌉₊)
#check (exists_nat_gt (1 / ε) : ∃ n : ℕ, 1 / ε < ↑n)
#check (lt_add_one a : a < a + 1)
#check (lt_of_le_of_lt : a ≤ b → b < c → a < c)
```

3.2.5. Demostración en lenguaje natural de la propiedad arquimediana

Teorema 3.2.2. *Para cualquier número real positivo $\varepsilon > 0$, existe un número natural N tal que $1/\varepsilon < N$.*

Demostración: Sea $\varepsilon > 0$. Necesitamos encontrar un número natural N tal que $1/\varepsilon < N$.

Usamos el valor $N = \lceil 1/\varepsilon \rceil_+ + 1$, donde $\lceil \cdot \rceil_+$ denota la función techo de números naturales.

Dado que $\varepsilon > 0$, tenemos $1/\varepsilon > 0$. Por la definición de la función techo de números naturales, sabemos que:

$$1/\varepsilon \leq \lceil 1/\varepsilon \rceil_+$$

Ahora, como $\lceil 1/\varepsilon \rceil_+$ es un número natural y $N = \lceil 1/\varepsilon \rceil_+ + 1$, tenemos:

$$\lceil 1/\varepsilon \rceil_+ < \lceil 1/\varepsilon \rceil_+ + 1 = N$$

Combinando estas desigualdades, obtenemos:

$$1/\varepsilon \leq \lceil 1/\varepsilon \rceil_+ < N$$

Por lo tanto, $1/\varepsilon < N$, lo que completa la demostración. □

Significado: La propiedad arquimediana es fundamental para el análisis porque asegura que los números reales no tienen elementos "infinitos" o "infinitesimales". Garantiza que siempre podemos encontrar números naturales lo suficientemente grandes como para dominar cualquier número real positivo dado cuando tomamos sus recíprocos. Esta propiedad es esencial para muchos procesos de límite y es equivalente a la completitud de los números reales en ciertas formulaciones.

3.2.6. Revisión de errores comunes

- ¡No uses la función techo regular $\lceil \cdot \rceil$! ¡Devuelve enteros, no naturales!
- **Cuidado con los problemas de casting**: si `bound` no funciona, prueba `push_cast` primero.
- La suma $\lceil 1/\varepsilon \rceil_+ + 1$ ocurre en $\mathbb{N}$, luego se convierte a $\mathbb{R}$; por eso necesitamos `push_cast`.

Nota histórica: Aunque a menudo se atribuye a Arquímedes (287–212 a.C.), esta propiedad probablemente era conocida por Eudoxo (408–355 a.C.) y aparece

en los *Elementos* de Euclides. Arquímedes utilizó una versión de este principio en su método de exhaución para calcular áreas y volúmenes.

3.3. Nivel 2: Nuestro primer límite real

¡Enhorabuena! Acabas de demostrar la propiedad arquimediana. Ahora vamos a usarla para demostrar algo realmente interesante: nuestro primer límite no trivial.

3.3.1. El objetivo: Demostrar que $1/n \to 0$

Queremos demostrar que la sucesión $a(n) = 1/n$ converge a 0 cuando n tiende a infinito. Esto es intuitivamente obvio: a medida que n crece, $1/n$ se hace más pequeño y se acerca a 0. Pero, ¿cómo lo hacemos riguroso usando la definición ε-N?

Teorema 3.3.1. *La sucesión $a(n) = 1/n$ converge a 0.*

3.3.2. Recordatorio: La definición de convergencia de sucesiones

Una sucesión $a : \mathbb{N} \to \mathbb{R}$ converge a un límite L (escrito `LimSuc a L`) si:

- Para cada $\varepsilon > 0$, existe $N \in \mathbb{N}$ tal que para todo $n \geq N$, $|a(n) - L| < \varepsilon$
- En notación formal: $\forall \varepsilon > 0, \exists N, \forall n \geq N, |a(n) - L| < \varepsilon$

Para nuestro caso específico con $a(n) = 1/n$ y $L = 0$, esto se convierte en: $\forall \varepsilon > 0, \exists N, \forall n \geq N, |1/n - 0| < \varepsilon$.

3.3.3. Estrategia de demostración en lenguaje natural

Así es como lo demostraremos paso a paso:

- **Paso 1:** Sea $\varepsilon > 0$ dado. (Esto corresponderá a `intro ε hε`).
- **Paso 2:** Necesitamos encontrar un N tal que para todo $n \geq N$, tengamos $1/n < \varepsilon$.
- **Idea clave:** Necesitamos $1/n < \varepsilon$, que es equivalente a $1/\varepsilon < n$ (ya que ambos lados son positivos). Así que necesitamos que n sea mayor que $1/\varepsilon$.
- **Paso 3:** ¡Para eso desarrollamos la propiedad arquimediana! Nos dice que existe algún número natural N tal que $1/\varepsilon < N$. En lugar de volver a demostrarlo desde cero, podemos simplemente citar este hecho; luego elegiremos tal N y lo usaremos.

- **Paso 4:** Ahora sea $n \geq N$ dado. Tenemos:
  - $1/\varepsilon < N$ (por nuestra elección de N)
  - $N \leq n$ (por hipótesis)
  - Por lo tanto: $1/\varepsilon < N \leq n$, así que $1/\varepsilon < n$
  - Tomando recíprocos (y volteando la desigualdad): $1/n < \varepsilon$
- **Paso 5:** Como $|1/n - 0| = |1/n| = 1/n < \varepsilon$, ¡hemos terminado!

3.3.4. Los desafíos de la implementación en Lean

3.3.4.1. Desafío 1: Multiplicación cruzada de fracciones

Nuestro paso clave es mostrar que $1/n < \varepsilon$. En matemáticas de papel, simplemente multiplicaríamos de forma cruzada para obtener $1 < n\varepsilon$. Pero Lean es muy cuidadoso con la división por cero. **Solución:** ¡La táctica `field_simp` maneja esto automáticamente! Eliminará denominadores multiplicando de forma cruzada, pero solo después de verificar que todos los denominadores son positivos (o al menos no nulos).

3.3.4.2. Desafío 2: Aritmética lineal

Una vez eliminadas las fracciones, necesitamos combinar varias desigualdades. La táctica `bound` no siempre maneja bien estas combinaciones lineales con multiplicación por variables. **Solución:** La táctica `linarith` está específicamente diseñada para la aritmética lineal. Puede tomar una lista de hipótesis y resolver metas que se sigan de combinaciones lineales de esas hipótesis.

3.3.4.3. Desafío 3: Casting explícito de tipos

¿Recuerdas las flechas hacia arriba ↑? ¡Han vuelto! Cuando escribimos $1/n$, Lean lo ve como $1/\uparrow n$ donde n comienza como un número natural pero necesita ser convertido a real. **Solución:** Podemos especificar el casting explícitamente con esta sintaxis: `(n : ℝ)`. Esto le dice a Lean exactamente a qué tipo queremos convertir n.

3.3.4.4. Desafío 4: Casting en aplicaciones de tácticas

A veces quieres aplicar una táctica pero los tipos no coinciden exactamente. **Solución:** La táctica `exact_mod_cas` es como `apply`, pero maneja automáticamente las coerciones de tipo por ti.

3.3.5. Nuevas herramientas que necesitarás

- `field_simp`: Elimina denominadores.
- `linarith`: Potente táctica para desigualdades lineales.
- `exact_mod_cast`: Maneja coerciones automáticamente cuando la meta es exactamente la hipótesis tras el casting.

3.3.6. Consejos profesionales para este nivel

1. Usa `change` para convertir `` `LimSuc a 0` `` a su definición.
2. Aplica la propiedad arquimediana para obtener la existencia de N.
3. Usa `choose` para extraer el N de la sentencia existencial.
4. Establece la positividad primero antes de usar `field_simp`.
5. Trabaja paso a paso.

3.3.7. Demostración en Lean

```
import Mathlib.Data.Real.Basic
import Mathlib.Tactic

variable (a : ℕ → ℝ)

def LimSuc (a : ℕ → ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, |a n - L| < ε

-- 1ª demostración
-- ===============

lemma L1
  {n : ℕ}
  : 0 ≤ 1 / (n : ℝ) :=
by
  -- ⊢ 0 ≤ 1 / ↑n
  apply div_nonneg
  · -- ⊢ 0 ≤ 1
    exact zero_le_one
  · -- ⊢ 0 ≤ ↑n
    exact Nat.cast_nonneg n

lemma L2
  {n : ℕ}
  : |1 / (n : ℝ)| = 1 / n :=
by
  apply abs_of_nonneg
  -- ⊢ 0 ≤ 1 / ↑n
  exact L1

variable {ε : ℝ}
variable {N : ℕ}

lemma L3
  (hε : ε > 0)
  (hN : 1 / ε < N)
```

```
  : 0 < (N : ℝ) :=
by calc
  (0 : ℝ) < 1 / ε := one_div_pos.mpr hε
  _       < N     := hN

lemma L4
  (hε : ε > 0)
  {n : ℕ}
  (hN : 1 / ε < N)
  (hn : n ≥ N)
  : 1 / (n : ℝ) ≤ 1 / (N : ℝ) :=
by
  apply one_div_le_one_div_of_le
  · -- ⊢ 0 < ↑N
    exact L3 hε hN
  · -- ⊢ ↑N ≤ ↑n
    exact Nat.cast_le.mpr hn

lemma L5
  (hε : ε > 0)
  (hN : 1 / ε < N)
  : 1 / (N : ℝ) < ε :=
by
  apply (one_div_lt _ _).mp
  · -- ⊢ 1 / ε < ↑N
    exact RCLike.ofReal_lt_ofReal.mp hN
  · -- ⊢ 0 < ε
    exact RCLike.ofReal_pos.mp hε
  · -- ⊢ 0 < ↑N
    exact L3 hε hN

example
  (ha : ∀ n, a n = 1 / n)
  : LimSuc a 0 :=
by
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |a n - 0| < ε
```

```
  have h1 : ∃ (N : ℕ), 1 / ε < N := exists_nat_gt (1 /
↪  ε)
  choose N hN using h1
  -- N : ℕ
  -- hN : 1 / ε < ↑N
  use N
  -- ⊢ ∀ n ≥ N, |a n - 0| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
  -- ⊢ |a n - 0| < ε
  calc |a n - 0|
       = |a n|          := by simp [sub_zero]
     _ = |1 / (n : ℝ)|  := by rw [ha]
     _ = 1 / n          := L2
     _ ≤ 1 / N          := L4 hε hN hn
     _ < ε              := L5 hε hN

-- 2ª demostración
-- ===============

lemma L2'
  {n : ℕ}
  : |1 / (n : ℝ)| = 1 / n :=
by
  apply abs_of_nonneg
  -- ⊢ 0 ≤ 1 / ↑n
  positivity

lemma L3'
  (hε : ε > 0)
  (hN : 1 / ε < N)
  : 0 < (N : ℝ) :=
by calc
  (0 : ℝ) < 1 / ε := by positivity
  _       < N     := hN

lemma L4'
  (hε : ε > 0)
  {n : ℕ}
```

```
  (hN : 1 / ε < N)
  (hn : n ≥ N)
  : 1 / (n : ℝ) ≤ 1 / (N : ℝ) :=
by
  apply one_div_le_one_div_of_le
  · -- ⊢ 0 < ↑N
    exact L3 hε hN
  · -- ⊢ ↑N ≤ ↑n
    gcongr

lemma L5'
  (hε : ε > 0)
  (hN : 1 / ε < N)
  : 1 / (N : ℝ) < ε :=
by
  apply (one_div_lt _ _).mp
  · -- ⊢ 1 / ε < ↑N
    gcongr
  · -- ⊢ 0 < ε
    gcongr
  · -- ⊢ 0 < ↑N
    exact L3 hε hN

example
  (ha : ∀ n, a n = 1 / n)
  : LimSuc a 0 :=
by
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |a n - 0| < ε
  have h1 : ∃ (N : ℕ), 1 / ε < N := exists_nat_gt (1 /
↪  ε)
  choose N hN using h1
  -- N : ℕ
  -- hN : 1 / ε < ↑N
  use N
  --⊢ ∀ n ≥ N, |a n - 0| < ε
  intro n hn
  -- n : ℕ
```

```
  -- hn : n ≥ N
  -- ⊢ |a n - 0| < ε
  calc |a n - 0|
       = |a n|          := by norm_num
     _ = |1 / (n : ℝ)|  := by rw [ha]
     _ = 1 / n          := L2'
     _ ≤ 1 / N          := L4' hε hN hn
     _ < ε              := L5' hε hN

-- Lemas usados
-- ============

variable (x y : ℝ)
variable (m n : ℕ)
#check (Nat.cast_le : ↑m ≤ ↑n ↔ m ≤ n)
#check (Nat.cast_nonneg n : 0 ≤ n)
#check (RCLike.ofReal_lt_ofReal : ↑x < ↑y ↔ x < y)
#check (abs_of_nonneg : 0 ≤ x → |x| = x)
#check (div_nonneg : 0 ≤ x → 0 ≤ y → 0 ≤ x / y)
#check (exists_nat_gt x : ∃ n : ℕ, x < n)
#check (one_div_le_one_div_of_le : 0 < x → x ≤ y → 1 /
 ↪  y ≤ 1 / x)
#check (one_div_lt : 0 < x → 0 < y → (1 / x < y ↔ 1 / y
 ↪  < x))
#check (one_div_pos : 0 < 1 / x ↔ 0 < x)
#check (zero_le_one : 0 ≤ 1)
```

3.3.8. Lo que acabamos de lograr

Esta demostración demuestra varios conceptos clave:

1. El poder de la propiedad arquimediana.
2. La definición ε-N en acción.
3. Manipulación rigurosa de desigualdades.

4. El puente entre la intuición y la formalidad.

4. No convergencia

4.1. Un gran debate

En el siglo XIX, cuando la gente aún tenía confusión sobre los límites en general, existía una sucesión concreta sobre la que se discutía acaloradamente:

$$1,\ -1,\ 1,\ -1,\ 1,\ -1,\ \ldots$$

¿Converge o no converge?

Algunos decían: subes uno y bajas uno, así que en promedio está alrededor de cero. Por tanto, el límite debería ser 0.

Otros respondían: fíjate en las sumas parciales de la sucesión:

$$\begin{aligned} 1 &= 1 \\ 1 + (-1) &= 0 \\ 1 + (-1) + 1 &= 1 \\ 1 + (-1) + 1 + (-1) &= 0 \end{aligned}$$

Las sumas parciales alternan entre 1 y 0, así que quizá el límite sea $\frac{1}{2}$.

Esto es el tipo de problema al que te enfrentas cuando no existe un consenso comunitario sobre las definiciones, es decir, cuando no hay una definición rigurosa de límite. ¡Por suerte, ahora sí la tenemos!

Con esa definición, esta sucesión … **no tiene límite** (diverge).

Para expresarlo formalmente, introduzcamos una nueva definición:

```
def SucConv (a : ℕ → ℝ) : Prop :=
  ∃ L, LimSuc a L
```

Es decir, una sucesión a «converge» (sin especificar a qué) si existe algún número real L tal que a converge a ese número.

El objetivo de este capítulo es demostrar:

$$\text{Meta} : \neg \texttt{SucConv}\ a,$$

dado la hipótesis

```
ha : ∀n, a n = (-1 : R)^n
```

(Nota: el símbolo ¬ se obtiene escribiendo `\not`.)

¿Cómo se demuestra la negación de algo? Lógicamente hablarías así: si eso ocurriera, obtendríamos una contradicción. Técnicamente, $\neg P$ es por definición equivalente a

$$P \to \text{False}$$

Por tanto, podemos comenzar la demostración recordándonos este hecho escribiendo:

```
change SucConv a → False
```

Eso cambiará la meta a: `SucConv a → False`.

En general, recomiendo el siguiente protocolo:

1. **Pensar.** Hacer un borrador, tener una idea vaga de cómo ensamblar el argumento.
2. **Demostrarlo formalmente.** ¡Pero no has terminado! Que algo funcione formalmente no significa necesariamente que entiendas qué está ocurriendo. El objetivo de las matemáticas *no* es que simplemente funcione, sino que **entiendas exactamente** qué está pasando y por qué. Por eso existe el paso 3:
3. **Dar una demostración en lenguaje natural** que explique (para ti mismo tanto como para cualquier otra persona, incluyéndome a mí) qué ocurre y por qué.

Usando únicamente reacciones instintivas (si ves $P \to Q$ en la meta, escribe `intro`; si ves $\exists$ en una hipótesis, escribe `choose`; si ves una definición en la meta o en una hipótesis, escribe `change`), llegamos al siguiente punto:

Objetos: $a : \mathbb{N} \to \mathbb{R}$, $L : \mathbb{R}$

Supuestos:

- `ha : ∀(n : N), a n = (-1)^n`
- `hL : ∀ε >0, ∃N, ∀n ≥ N, |a n - L| <ε`

Meta: `False`

Recuerda al Ingeniero y al Mecánico: el Ingeniero especifica cualquier tolerancia, y el Mecánico debe garantizar que estaremos dentro de esa tolerancia «eventualmente», es decir, para todo n suficientemente grande, medido por la cota inferior N.

Idea clave: Encontrar un ε cuya tolerancia sea demasiado ajustada para que el Mecánico pueda cumplirla. Como la sucesión alterna entre 1 y -1, que difieren en 2, la tolerancia $\varepsilon = 1$ ya sería suficiente, pero para tener un poco más de margen tomemos $\varepsilon = \frac{1}{2}$.

Nota 4.1.1. Si hubiéramos tomado $\varepsilon = 1{,}5$, *no* funcionaría. No podemos descartar la posibilidad de que $L = 0$, y los valores 1 y -1 *sí* están dentro de esa tolerancia. Por tanto, no hallaríamos ninguna contradicción.

Tras varios cálculos, llegamos a:

$$|1 - L| < \tfrac{1}{2} \qquad \text{y} \qquad |-1 - L| < \tfrac{1}{2}.$$

Intentemos obtener una contradicción a partir de ahí:

$$\begin{aligned}
2 &= |2| \\
&= |1 - (-1)| \\
&= |(1 - L) + (L - (-1))| \\
&\leq |(1 - L)| + |(L - (-1))| \\
&= |(1 - L)| + |-((-1) - L)| \\
&= |(1 - L)| + |((-1) - L)| \\
&< \tfrac{1}{2} + \tfrac{1}{2} \\
&= 1.
\end{aligned}$$

Eso nos da la contradicción deseada.

Nuevas herramientas que necesitarás

- **Negación:** Si P es alguna `Prop`, entonces $\neg P$ es por definición equivalente a $P \to$ False. Por tanto, puedes escribir ‘`change P → False`‘, ya sea en la meta o en una hipótesis.
- **La desigualdad triangular:** Para añadir el hecho de que $|x + y| \leq |x| + |y|$ a nuestra lista de hipótesis, invoca el teorema `abs_add`:

```
have h : |x + y| ≤ |x| + |y| := by apply abs_add
```

- **Negación dentro de un valor absoluto:** También puede resultar útil el teorema `abs_neg`, que se llama así:

```
have h : |-x| = |x| := by apply abs_neg
```

¡Atención! Asegúrate de que el patrón `|-Algo|` esté en el lado izquierdo. Si Lean no ve un valor absoluto seguido de un signo menos, `abs_neg` no funcionará.

4.2. Paso 2: Solución en Lean

```
-- Demostración en lenguaje natural
-- ================================

-- Sea a la sucesión definida por a(n) = (-1)^n.
 ↪  Supongamos que a es
-- convergente. Entonces, existe un L tal que a
 ↪  converge L. Por tanto,
-- existe un N ∈ ℕ tal que,
--    ∀n ≥ N, |a(n) - L| < 1/2
 ↪  (1)
-- Entonces,
--    2 = |2|
--      = |(1 - L) + (1 + L)|
--      = |(1 - L) + (-1)(-1 - L)|
--      ≤ |1 - L| + |(-1)(-1 - L)|
--      = |1 - L| + |-1 - L|
--      = |(-1)^{2N} - L| + |(-1)^{2N+1} - L|
--      = |a(2N) - L| + |a(2N+1) - L|
--      < 1/2 + 1/2                              [por (1),
 ↪  2N ≥ N y 2N+1 ≥ N]
--      = 1
-- Luego, 2 < 1 que es una contradicción.

-- 1ª demostración
```

```
-- ===============

example
  (ha : ∀ n, a n = (-1) ^ n)
  : ¬ SucConv a :=
by
  intro h
  -- h : SucConv a
  -- ⊢ False
  choose L hL using h
  -- L : ℝ
  -- hL : LimSuc a L
  choose N hN using hL (1/2) (by grind)
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < 1 / 2
  have h1 : (2:ℝ) < 1 := by calc
    2 = |2|                                                  :=
↪   by grind
    _ = |(1 - L) + (1 + L)|                                  :=
↪   by grind
    _ = |(1 - L) + (-1)*(-1 - L)|                            :=
↪   by grind
    _ ≤ |1 - L| + |(-1)*(-1 - L)|                            :=
↪   by grind
    _ = |1 - L| + |-1 - L|                                   :=
↪   by grind
    _ = |(-1:ℝ)^(2*N) - L| + |(-1:ℝ)^(2*N+1) - L|            :=
↪   by
          have h2 : (-1:ℝ)^(2*N) = 1    := by simp
          have h3 : (-1:ℝ)^(2*N+1) = -1 := by grind
          rw [h2, h3]
    _ = |a (2*N) - L| + |a (2*N+1) - L|                      :=
↪   by simp [ha]
    _ < 1/2 + 1/2                                            :=
↪   by grind
    _ = 1                                                    :=
↪   by grind
  linarith

-- 2ª demostración
```

```
-- ===============

variable {x y z x' y' : ℝ}
variable {m n k : ℕ}
variable (f : ℝ → ℝ)

lemma L1 : (1 - x) + (1 + x) = 2 := by
  calc (1 - x) + (1 + x)
       = 1 + 1             := sub_add_add_cancel 1 1 x
     _ = 2                 := one_add_one_eq_two

lemma L2 : (-1:ℝ) * -1 = 1 := by
  calc (-1:ℝ) * -1
       = 1 * 1   := neg_mul_neg 1 1
     _ = 1       := one_mul 1

lemma L3 : -1 * -x = x := by
  calc -1 * -x
       = -(1 * -x) := neg_mul 1 (-x)
     _ = - -x      := congr_arg (- ·) (one_mul (-x))
     _ = x         := neg_neg x

lemma L4 : -1 * (-1 - x) = 1 + x := by
  calc -1 * (-1 - x)
       = -1 * (-1 + -x)     := congr_arg (-1 * ·)
↪  (sub_eq_add_neg (-1) x)
     _ = -1 * -1 + -1 * -x  := left_distrib (-1) (-1)
↪  (-x)
     _ = 1 + -1 * -x        := congr_arg (· +
↪  (-1)*(-x)) L2
     _ = 1 + x              := congr_arg ( 1 + ·) L3

lemma L5 : |-1 * (-1 - x)| = |-1 - x| := by
  calc |-1 * (-1 - x)|
       = |-1| * |-1 - x| := abs_mul (-1) (-1 - x)
     _ = |1| * |-1 - x|  := congrArg (· * |-1 - x|)
↪  (abs_neg 1)
     _ = 1 * |-1 - x|    := congrArg (· * |-1 - x|)
↪  abs_one
     _ = |-1 - x|        := one_mul |-1 - x|
```

```
lemma L6 : (-1:ℝ)^(2*n) = 1 := by
  calc (-1:ℝ)^(2*n)
       = ((-1:ℝ)^2)^n := pow_mul (-1) 2 n
     _ = (1:ℝ)^n      := congr_arg ( · ^ n) neg_one_sq
     _ = 1            := one_pow n

lemma L7 : (-1:ℝ)^(2*n+1) = -1 := by
  calc (-1:ℝ)^(2*n+1)
       = (-1)^(2*n) * -1 := pow_succ (-1) (2 * n)
     _ = 1 * -1          := congr_arg (· * -1) L6
     _ = -1              := one_mul (-1)

lemma L8 : n ≤ 2 * n := by
  calc n
       = 1 * n := (one_mul n).symm
     _ ≤ 2 * n := Nat.mul_le_mul_right n one_le_two

lemma L9
  (hN : ∀ n ≥ N, |a n - x| < 1 / 2)
  : |a (2*N) - x| < 1 / 2 :=
hN (2*N) L8

lemma L10 : n ≤ 2 * n + 1 := by
  calc n
       ≤ 2 * n     := L8
     _ ≤ 2 * n + 1 := Nat.le_add_right (2 * n) 1

lemma L11
  (hN : ∀ n ≥ N, |a n - x| < 1 / 2)
  : |a (2*N+1) - x| < 1 / 2 :=
hN (2*N+1) L10

example
  (ha : ∀ n, a n = (-1) ^ n)
  : ¬ SucConv a :=
by
  intro h
  -- h : SucConv a
  -- ⊢ False
```

```
  choose L hL using h
  -- L : ℝ
  -- hL : LimSuc a L
  choose N hN using hL (1/2) one_half_pos
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < 1 / 2
  have h1 : (2:ℝ) < 1 := by calc
    2 = |2| :=
          abs_two.symm
    _ = |(1 - L) + (1 + L)| :=
          congrArg abs L1.symm
    _ = |(1 - L) + (-1)*(-1 - L)| :=
          congr_arg (abs ((1 - L) + ·)) L4.symm
    _ ≤ |1 - L| + |(-1)*(-1 - L)| :=
          abs_add_le (1 - L) (-1 * (-1 - L))
    _ = |1 - L| + |-1 - L| :=
          congr_arg (|1 - L| + ·) L5
    _ = |(-1:ℝ)^(2*N) - L| + |(-1:ℝ)^(2*N+1) - L|  :=
          congrArg₂ (|· - L| + |· - L|) L6.symm L7.symm
    _ = |a (2*N) - L| + |a (2*N+1) - L| :=
          congrArg₂ (· + ·)
            (congrArg (|· - L|) (ha (2*N)).symm)
            (congrArg (|· - L|) (ha (2*N+1)).symm)
    _ < 1 / 2 + 1 / 2 :=
          add_lt_add (L9 hN) (L11 hN)
    _ = 1 := add_halves 1
  have h2 : (1:ℝ) < 1 := lt_trans one_lt_two h1
  exact (lt_irrefl 1) h2

-- Lemas usados
-- ============

#check (Nat.le_add_right n m : n ≤ n + m)
#check (Nat.mul_le_mul_right k : n ≤ m → n * k ≤ m * k)
#check (abs_add_le x y : |x + y| ≤ |x| + |y|)
#check (add_halves x : x / 2 + x / 2 = x)
#check (add_lt_add : x < x' → y < y' → x + y < x' + y')
#check (congrArg₂  (· + ·) : x = x' → y = y' → x + y =
 ↪  x' + y')
#check (congr_arg f  : x = y → f x = f y)
```

```
#check (left_distrib x y z : x * (y + z) = x * y + x *
 ↪  z)
#check (lt_irrefl x : ¬x < x)
#check (lt_trans : x < y → y < z → x < z)
#check (neg_mul 1 x : -1 * x = -(1 * x))
#check (neg_mul_neg x y : -x * -y = x * y)
#check (neg_neg x : - -x = x)
#check (neg_one_sq : (-1)^2 = 1)
#check (one_add_one_eq_two : 1 + 1 = 2)
#check (one_le_two : 1 ≤ 2)
#check (one_lt_two : (1 : ℝ) < 2)
#check (one_mul x : 1 * x = x)
#check (one_mul x : 1 * x = x)
#check (one_pow n : 1 ^ n = 1)
#check (pow_mul x m n : x ^ (m * n) = (x ^ m) ^ n)
#check (pow_succ x n : x ^ (n + 1) = x ^ n * x)
#check (sub_add_add_cancel x y z : (x - z) + (y + z) =
 ↪  x + y)
#check (sub_eq_add_neg x y : x - y = x + -y)

-- 3ª demostración
-- ===============

example
  (ha : ∀ n, a n = (-1) ^ n)
  : ¬ SucConv a :=
by
  change SucConv a → False
  -- ⊢ SucConv a → False
  intro h
  -- h : SucConv a
  -- ⊢ False
  change ∃ L, LimSuc a L at h
  -- h : ∃ L, LimSuc a L
  choose L hL using h
  -- L : ℝ
  -- hL : LimSuc a L
  change ∀ ε > 0, ∃ N, ∀ n ≥ N, |a n - L| < ε at hL
  -- hL : ∀ ε > 0, ∃ N, ∀ n ≥ N, |a n - L| < ε
  specialize hL (1/2)
```

```
  -- hL : 1 / 2 > 0 → ∃ N, ∀ n ≥ N, |a n - L| < 1 / 2
  have h0 : (0 : ℝ) < 1/2 := by norm_num
  specialize hL h0
  -- hL : ∃ N, ∀ n ≥ N, |a n - L| < 1 / 2
  choose N hN using hL
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < 1 / 2
  have h2N : N ≤ 2 * N := by bound
  have h2Np1 : N ≤ 2 * N + 1 := by bound
  have h1 : |a (2 * N) - L| < 1 / 2 := by apply hN (2 *
↪  N) h2N
  have h2 : |a (2 * N + 1) - L| < 1 / 2 := by apply hN
↪  (2 * N + 1) h2Np1
  have h3 : a (2 * N) = (-1) ^ (2 * N) := by apply ha
↪  (2 * N)
  have h4 : a (2 * N + 1) = (-1) ^ (2 * N + 1) := by
↪  apply ha (2 * N + 1)
  rewrite [h3] at h1
  -- h1 : |(-1) ^ (2 * N) - L| < 1 / 2
  rewrite [h4] at h2
  -- h2 : |(-1) ^ (2 * N + 1) - L| < 1 / 2
  have h5 : (-1 : ℝ) ^ (2 * N) = 1 := by bound
  rewrite [h5] at h1
  -- h1 : |1 - L| < 1 / 2
  have h6 : (-1 : ℝ) ^ (2 * N + 1) = -1 := by grind
  rewrite [h6] at h2
  -- h2 : |-1 - L| < 1 / 2
  have h7 : (2 : ℝ) = |2| := by norm_num
  have h8 : |(2 : ℝ)| = |1 - (-1)| := by ring_nf
  have h9 : |1 - (-1)| = |(1 - L) + (L - (-1))| := by
↪  noncomm_ring
  have h10 : |(1 - L) + (L - (-1))| ≤ |(1 - L)| + |(L -
↪  (-1))| :=
    abs_add_le (1 - L) (L - -1)
  have h11 : |(L - (-1))| = |-((-1) - L)| := by ring_nf
  have h12 : |-((-1) - L)| = |((-1) - L)| := by apply
↪  abs_neg
  have h13 : (2 : ℝ) < 1/2 + 1/2 := by
    linarith [h8, h9, h10, h11, h12, h7, h1, h2]
  norm_num at h13
```

4.3. Paso 3: Demostración en lenguaje natural

4.3.1. Demostración en lenguaje natural

Teorema 4.3.1. *La sucesión* $1, -1, 1, -1, \ldots$ *no converge.*

Demostración por contradicción.

1. Supongamos que la sucesión converge a algún límite L.
2. Tomamos $\varepsilon = \frac{1}{2}$ y aplicamos la definición de convergencia.
3. Esto garantiza la existencia de algún N tal que $|a_n - L| < \frac{1}{2}$ para todo $n \geq N$.
4. Consideramos dos índices concretos: $n = 2N$ y $n = 2N + 1$ (ambos son $\geq N$):
   - $a(2N) = (-1)^{2N} = 1$, luego $|1 - L| < \frac{1}{2}$.
   - $a(2N + 1) = (-1)^{2N+1} = -1$, luego $|-1 - L| < \frac{1}{2}$.

5. Pero entonces:

$$\begin{aligned} 2 &= |1-(-1)| \\ &= |(1-L)+(L+1)| \\ &\leq |1-L|+|L+1| \quad \text{(desigualdad triangular)} \\ &= |1-L|+|-1-L| \quad \text{(manipulación algebraica)} \\ &< \tfrac{1}{2}+\tfrac{1}{2} \\ &= 1 \quad \text{(de los pasos anteriores).} \end{aligned}$$

6. Esto nos da $2 < 1$, lo cual es imposible. □

4.3.2. Ideas clave

- **Intuición geométrica:** Cualquier límite propuesto L debe estar simultáneamente a distancia menor de $\frac{1}{2}$ tanto de 1 como de -1, pero estos valores están a distancia 2 entre sí.
- **Elección crítica:** Escogimos $\varepsilon = \frac{1}{2}$ de forma estratégica; valores mayores como $\varepsilon = 1{,}5$ no funcionarían, ya que $L = 0$ podría satisfacer las restricciones.
- **Desigualdad triangular:** El paso clave usa $|a+b| \leq |a|+|b|$ para convertir la distancia entre los valores de la sucesión en una suma de distancias al límite.

Esta demostración ilustra cómo las definiciones rigurosas resuelven debates matemáticos históricos y proporcionan criterios claros para la convergencia.

5. Duplicar una sucesión convergente

5.1. El desafío de escalar en la fábrica

Tras conquistar la sucesión constante, subamos el nivel: si una sucesión converge, entonces duplicar esa sucesión también converge, y converge al doble del límite original.

Imaginemos que el Maquinista recibe un encargo del Ingeniero: «Por favor, duplica todas las longitudes, pero mantén los mismos estándares de calidad». ¿Cómo debería responder el Maquinista?

Si el Ingeniero exige que las longitudes duplicadas estén dentro de ε de $2L$, el Maquinista no puede simplemente pedir que el proceso original cumpla la tolerancia original ε, porque

$$|2a(n) - 2L| < 2\varepsilon,$$

no ε. En cambio, debemos ser más ingeniosos. ¿Se te ocurre qué hacer?

Exacto: el ε del proceso original es arbitrario, ¡así que podemos jugar con él! Si logramos que el proceso original garantice una salida dentro de $\varepsilon/2$ de L, entonces

al duplicar esa salida estaremos dentro de ε de $2L$.

Idea clave: al escalar por una constante, debemos escalar las exigencias de tolerancia de forma inversa.

5.2. La configuración matemática

Supongamos que tenemos:

- Una sucesión $a : \mathbb{N} \to \mathbb{R}$ que converge a L.
- Una sucesión $b : \mathbb{N} \to \mathbb{R}$ con la propiedad de que $b(n) = 2a(n)$ para todo n.

Queremos demostrar que b converge a $2L$.

5.2.1. Idea Clave: Escalado Inverso de la Tolerancia

La observación crucial es que:

$$|b(n) - 2L| = |2a(n) - 2L| = 2|a(n) - L|.$$

Así, si queremos $|b(n) - 2L| < \varepsilon$, necesitamos $2|a(n) - L| < \varepsilon$, es decir:

$$|a(n) - L| < \frac{\varepsilon}{2}.$$

¡Esto es exactamente lo que nos proporciona la convergencia de a!

5.3. Nuevas herramientas que necesitarás

5.3.1. Valor absoluto de productos

Necesitarás el nuevo teorema `abs_mul`, que establece que para cualesquiera números reales x e y:

$$|xy| = |x||y|.$$

Para usar este teorema puede resultar conveniente crear una nueva hipótesis con `have` y luego reescribir con esa hipótesis. Es decir, puedes escribir:

```
have NuevoHecho : | Algo * OtraCosa | =
    | Algo | * | OtraCosa | :=
  by apply abs_mul
```

y después `rewrite [NuevoHecho]` sustituirá `|Algo * OtraCosa|` por `|Algo| * |OtraCosa|` (ya sea en el Objetivo o en una hipótesis, si así se especifica).

5.4. Enfoque estratégico

1. Despliega la definición de convergencia para el objetivo.

2. Dado cualquier $\varepsilon > 0$, usa la convergencia de a con tolerancia $\varepsilon/2$. Puede ser útil demostrar por

separado la desigualdad $0 < \varepsilon/2$: ¿qué táctica crees que ayudaría aquí?

3. Extrae el testigo N de la convergencia de a.

4. Usa el mismo N para tu sucesión b.

5. Aplica la hipótesis de escalado y usa `abs_mul` para relacionar $|b(n) - 2L|$ con $|a(n) - L|$.

6. Usa la cota de convergencia de a para concluir.

5.5. Solución en Lean

```
-- Demostración en lenguaje natural
-- ================================

-- Sea bₙ la sucesión definida por
--    bₙ = 2aₙ
 ↪ (1)
-- Tenemos que demostrar que para cada ε > 0, existe un
 ↪ N ∈ ℕ tal que
--    ∀ n ≥ N, |bₙ - 2L| < ε
 ↪ (2)
-- Puesto que aₙ converge a L, existe un N ∈ ℕ tal que
--    ∀ n ≥ N, |aₙ - L| < ε/2
 ↪ (3)
-- Veamos que N también cumple (2). En efecto, sea n ≥
 ↪ N. Entonces
--    |bₙ - 2L| = |2aₙ - 2L|     [por (1)]
--              = 2|aₙ - L|
--              < 2ε/2           [por (3)]
--              = ε

-- Demostraciones con Lean 4
```

```
-- ========================

import Mathlib.Data.Real.Basic
import Mathlib.Tactic
import Analisis_en_Lean.C2_El_calculo_de_pi

variable {a b : ℕ → ℝ}
variable {L : ℝ}

-- 1ª demostración
-- ===============

example
  (ha : LimSuc a L)
  (hb : ∀ n, b n = 2 * a n)
  : LimSuc b (2 * L) :=
by
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |b n - 2 * L| < ε
  obtain ⟨N, hN⟩ := ha (ε / 2) (by grind)
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < ε / 2
  use N
  -- ⊢ ∀ n ≥ N, |b n - 2 * L| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
  -- ⊢ |b n - 2 * L| < ε
  grind

-- 2ª demostración
-- ===============

example
  (ha : LimSuc a L)
  (hb : ∀ n, b n = 2 * a n)
  : LimSuc b (2 * L) :=
by
```

```
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |b n - 2 * L| < ε
  obtain ⟨N, hN⟩ := ha (ε / 2) (by grind)
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < ε / 2
  use N
  -- ⊢ ∀ n ≥ N, |b n - 2 * L| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
  -- ⊢ |b n - 2 * L| < ε
  calc |b n - 2 * L|
       = |2 * a n - 2 * L| := by grind
     _ = |2 * (a n - L)|   := by grind
     _ = 2 * |a n - L|     := by grind
     _ < 2 * ε / 2         := by grind
     _ = ε                 := by grind

-- 3ª demostración
-- ===============

example
  (ha : LimSuc a L)
  (hb : ∀ n, b n = 2 * a n)
  : LimSuc b (2 * L) :=
by
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |b n - 2 * L| < ε
  obtain ⟨N, hN⟩ := ha (ε / 2) (half_pos hε)
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < ε / 2
  use N
  -- ⊢ ∀ n ≥ N, |b n - 2 * L| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
```

```
  -- ⊢ |b n - 2 * L| < ε
  calc |b n - 2 * L|
       = |2 * a n - 2 * L| := by rw [hb n]
     _ = |2 * (a n - L)|   := by simp only [mul_sub]
     _ = |2| * |a n - L|   := by simp only [abs_mul]
     _ = 2 * |a n - L|     := by simp only [abs_two]
     _ < 2 * (ε / 2)       := by simp only [
↪  mul_lt_mul_of_pos_left, hN n hn, two_pos]
     _ = ε                 := by rw [mul_div_cancel₀ ε
↪  two_ne_zero]

-- 4ª demostración
-- ===============

example
  (ha : LimSuc a L)
  (hb : ∀ n, b n = 2 * a n)
  : LimSuc b (2 * L) :=
by
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |b n - 2 * L| < ε
  obtain ⟨N, hN⟩ := ha (ε / 2) (half_pos hε)
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < ε / 2
  use N
  -- ⊢ ∀ n ≥ N, |b n - 2 * L| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
  -- ⊢ |b n - 2 * L| < ε
  calc |b n - 2 * L|
       = |2 * a n - 2 * L| := congr_arg (|· - 2 * L|)
↪  (hb n)
     _ = |2 * (a n - L)|   := congr_arg abs (mul_sub 2
↪  (a n) L).symm
     _ = |2| * |a n - L|   := abs_mul 2 (a n - L)
     _ = 2 * |a n - L|     := congr_arg (· * |a n - L|)
↪  abs_two
```

```
    _ < 2 * (ε / 2)       := mul_lt_mul_of_pos_left
↪  (hN n hn) two_pos
    _ = ε                 := mul_div_cancel₀ ε
↪  two_ne_zero

-- Lemas usados
-- ============

variable (x y z : ℝ)

#check (abs_mul x y : |x * y| = |x| * |y|)
#check (abs_two : |(2 : ℝ)| = 2)
#check (half_pos : 0 < x → 0 < x / 2)
#check (mul_div_cancel₀ x : y ≠ 0 → y * (x / y) = x)
#check (mul_lt_mul_of_pos_left : x < y → 0 < z → z * x
↪  < z * y)
#check (mul_sub x y z : x * (y - z) = x * y - x * z)
#check (two_ne_zero : 2 ≠ 0)
#check (zero_lt_two : 0 < 2)

-- 3ª demostración
-- ===============

example
  (ha : LimSuc a L)
  (hb : ∀ n, b n = 2 * a n)
  : LimSuc b (2 * L) :=
by
  change ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, |b n - 2 * L| < ε
  -- ⊢ ε > 0 → ∃ N, ∀ n ≥ N, |b n - 2 * L| < ε
  intro ε hε
  -- ε : ℝ
  -- hε : ε > 0
  -- ⊢ ∃ N, ∀ n ≥ N, |b n - 2 * L| < ε
  change ∀ ε₁ > 0, ∃ N₁ : ℕ, ∀ n ≥ N₁, |a n - L| < ε₁
↪  at ha
  -- ha : ∀ ε₁ > 0, ∃ N₁, ∀ n ≥ N₁, |a n - L| < ε₁
  specialize ha (ε / 2)
  -- ha : ε / 2 > 0 → ∃ N₁, ∀ n ≥ N₁, |a n - L| < ε / 2
  have hε2 : 0 < ε / 2 := by bound
```

```
  specialize ha hε2
  -- ha : ∃ N₁, ∀ n ≥ N₁, |a n - L| < ε / 2
  choose N hN using ha
  -- N : ℕ
  -- hN : ∀ n ≥ N, |a n - L| < ε / 2
  use N
  -- ⊢ ∀ n ≥ N, |b n - 2 * L| < ε
  intro n hn
  -- n : ℕ
  -- hn : n ≥ N
  specialize hb n
  -- hb : b n = 2 * a n
  rewrite [hb]
  -- ⊢ |2 * a n - 2 * L| < ε
  clear hb hε2
  have h1 : 2 * a n - 2 * L = 2 * (a n - L) := by
↪  ring_nf
  rewrite [h1]
  -- ⊢ |2 * (a n - L)| < ε
  clear h1
  have h2 : |2 * (a n - L)| = |2| * |a n - L| := by
↪  apply abs_mul
  rewrite [h2]
  -- ⊢ |2| * |a n - L| < ε
  clear h2
  specialize hN n hn
  -- hN : |a n - L| < ε / 2
  clear hn
  norm_num
  -- ⊢ 2 * |a n - L| < ε
  bound
```

5.6. Prueba en lenguaje natural

Demos un paso atrás respecto a la prueba formal en Lean y comprendamos en castellano lo que acabamos

de demostrar.

5.6.1. Enunciado del teorema

Teorema 5.6.1. *Si una sucesión de números reales converge a algún límite, entonces la sucesión formada al duplicar cada término converge al doble del límite original.*

5.6.2. Prueba

Demostración: Supongamos que la sucesión a_n converge a L, y queremos demostrar que $b_n = 2a_n$ converge a $2L$.

Por definición, debemos demostrar que para cualquier tolerancia $\varepsilon > 0$, podemos encontrar un punto N tal que para todo $n \geq N$ se tenga $|b_n - 2L| < \varepsilon$.

La idea clave es la siguiente: como a_n converge a L, podemos hacer $|a_n - L|$ arbitrariamente pequeño. Concretamente, podemos encontrar un N tal que $|a_n - L| < \varepsilon/2$ para todo $n \geq N$.

Ahora bien, para cualquier $n \geq N$:

$$|b_n - 2L| = |2a_n - 2L| \tag{5.1}$$
$$= |2(a_n - L)| \tag{5.2}$$
$$= 2|a_n - L| \tag{5.3}$$
$$< 2\frac{\varepsilon}{2} \tag{5.4}$$
$$= \varepsilon \tag{5.5}$$

Por tanto, b_n converge a $2L$, completando la prueba. □

5.7. Por qué importa este patrón

Esta prueba introduce la importante técnica del *escalado inverso de tolerancias*. Cuando se escala una sucesión por una constante c, es necesario escalar las exigencias de tolerancia por $1/c$. Este principio reaparecerá cuando estudies:

- Productos de sucesiones (donde ambos factores contribuyen al error).
- Derivadas (donde la definición de límite involucra escalado por h).
- Integración (donde las sumas de Riemann involucran escalado por los anchos de partición).

¡La capacidad de gestionar cómo las constantes afectan a la convergencia es fundamental en todo el análisis!

5.7.1. La estrategia elegante

La prueba utilizó el *principio de escalado inverso*:

1. **Inversión de la tolerancia:** para lograr tolerancia ε en la salida duplicada, se exige tolerancia $\varepsilon/2$ en la entrada original.

2. **Factorización algebraica:** $2a_n - 2L = 2(a_n - L)$ reveló la estructura.

3. **Escalado del valor absoluto:** $|2x| = |2||x|$, seguido de `norm_num`, convirtió la forma factorizada en la cota necesaria.

4. **Aritmética lineal:** la cota final combinó $2|a_n - L| < 2(\varepsilon/2) = \varepsilon$.

6. Unicidad y teoremas avanzados de límites

6.1. Unicidad de límites

Una de las propiedades fundamentales de las sucesiones convergentes es que convergen a un límite único. Esto puede parecer obvio a primera vista: ¿cómo podría una sucesión acercarse arbitrariamente a dos números distintos? Sin embargo, como ocurre con muchos hechos intuitivos en análisis, la prueba rigurosa requiere un razonamiento cuidadoso con nuestras definiciones épsilon-N.

La clave para demostrar la unicidad es la *prueba por contradicción*. Asumiremos que una sucesión converge a dos límites distintos L y M, y mostraremos que esto lleva a una imposibilidad. La estrategia consiste en elegir épsilon igual a la mitad de la distancia entre L y M, y luego demostrar que la sucesión no puede permanecer tan cerca de ambos límites simultáneamente.

6.1.1. Nuevas herramientas

6.1.1.1. Prueba por contradicción: `by_contra`

La táctica `by_contra` nos permite probar un enunciado asumiendo su negación y derivando una contradicción. La sintaxis es `by_contra h`, que añade una hipótesis `h` con la negación del objetivo actual y cambia el objetivo a `False`.

6.1.1.2. Positividad del valor absoluto: `abs_pos_of_no`

El teorema `abs_pos_of_nonzero` establece que si $x \neq 0$, entonces $0 < |x|$. Esto es esencial para trabajar con distancias entre puntos distintos.

6.1.2. Enunciado matemático

Teorema 6.1.1 (Unicidad del límite)**.** *Si una sucesión* $a : \mathbb{N} \to \mathbb{R}$ *converge tanto a* L *como a* M*, entonces* $L = M$*.*

6.1.3. Demostración en Lean

```
theorem LimUnique (a : ℕ → ℝ) (L M : ℝ)
    (aToL : SeqLim a L) (aToM : SeqLim a M) :
    L = M := by
  by_contra h
  have f0 : L - M ≠ 0 := by bound
```

```
  have f1 : 0 < |L - M| := by apply
↪  abs_pos_of_nonzero f0
  have f2 : 0 < |L - M| / 2 := by bound
  specialize aToL (|L - M| / 2) f2
  specialize aToM (|L - M| / 2) f2
  choose N1 hN1 using aToL
  choose N2 hN2 using aToM
  have f3 : N1 ≤ N1 + N2 := by bound
  have f4 : N2 ≤ N1 + N2 := by bound
  specialize hN1 (N1 + N2) f3
  specialize hN2 (N1 + N2) f4
  have f5 : |L - M| =
    |(L - a (N1 + N2)) + (a (N1 + N2) - M)| :=
↪  by ring_nf
  have f6 : |(L - a (N1 + N2)) + (a (N1 + N2) -
↪  M)| ≤
    |(L - a (N1 + N2))| + |(a (N1 + N2) - M)| :=
↪  by apply abs_add
  have f7 : |(L - a (N1 + N2))| =
    |-(a (N1 + N2) - L)| := by ring_nf
  have f8 : |-(a (N1 + N2) - L)| =
    |(a (N1 + N2) - L)| := by apply abs_neg
  linarith [f5, f6, f7, f8, hN1, hN2]
```

6.1.4. Prueba en lenguaje natural

Demostración: Supongamos, por contradicción, que $L \neq M$. Entonces $L - M \neq 0$, por lo que $|L - M| > 0$, y por tanto $\varepsilon := |L - M|/2 > 0$.

Como $a(n) \to L$, existe N_1 tal que $|a(n) - L| < \varepsilon$ para todo $n \geq N_1$. Como $a(n) \to M$, existe N_2 tal que $|a(n) - M| < \varepsilon$ para todo $n \geq N_2$.

Sea $N = N_1 + N_2$. Para $n = N$ tenemos simultáneamente $|a(N) - L| < \varepsilon$ y $|a(N) - M| < \varepsilon$. Observemos que:

$$|L - M| = |(L - a(N)) + (a(N) - M)|.$$

Por la desigualdad triangular:

$$|L-M| \leq |L-a(N)|+|a(N)-M| = |a(N)-L|+|a(N)-M| < \varepsilon+\varepsilon$$

Esto nos da $|L - M| < |L - M|$, lo cual es una contradicción. Por tanto, $L = M$. □

6.1.5. Por qué importa la unicidad

La unicidad de los límites es fundamental para todo el edificio del análisis. Nos permite hablar de *el* límite de una sucesión en lugar de *un* límite, y garantiza que nuestra noción de convergencia está bien definida. Sin la unicidad, muchos teoremas y técnicas estándar fallarían o requerirían modificaciones sustanciales.

6.2. Eventualmente: Las sucesiones convergentes permanecen cerca de su límite

Cuando una sucesión converge a un límite distinto de cero, no solo se acerca arbitrariamente a ese límite,

sino que *eventualmente* también se aleja de cero. Esta propiedad de «estar eventualmente acotada lejos de cero» es crucial para muchos teoremas que involucran cocientes y recíprocos.

La intuición es sencilla: si una sucesión converge a algún valor no nulo L, entonces en algún momento los términos de la sucesión deben tener valor absoluto al menos la mitad del de L. No pueden simultáneamente acercarse a L y reducirse hacia cero.

6.2.1. Enunciado matemático

Teorema 6.2.1. *Si $a : \mathbb{N} \to \mathbb{R}$ converge a L con $L \neq 0$, entonces existe N tal que para todo $n \geq N$ se tiene $|a(n)| \geq |L|/2$.*

6.2.2. Enfoque estratégico

La clave está en usar la condición de convergencia con $\varepsilon = |L|/2$. Como $L \neq 0$, tenemos $|L| > 0$, por lo que este épsilon es positivo. La condición de convergencia nos dice entonces que eventualmente $|a(n) - L| < |L|/2$, lo que por la desigualdad triangular inversa implica que $|a(n)|$ es al menos $|L|/2$.

6.2.3. Demostración en Lean

```
theorem EventuallyGeHalfLim (a : ℕ → ℝ) (L : ℝ)
    (aToL : SeqLim a L) (LneZero : L ≠ 0) :
    ∃ N, ∀ n ≥ N, |L| / 2 ≤ |a n| := by
  specialize aToL (|L| / 2)
  have : 0 < |L| := by apply abs_pos_of_nonzero
↪  LneZero
  have : 0 < |L| / 2 := by bound
  specialize aToL this
  choose N hN using aToL
  use N
  intro n hn
  specialize hN n hn
  have l1 : |L| = |a n + (L - a n)| := by
↪  ring_nf
  have l2 : |a n + (L - a n)| ≤ |a n| + |L - a
↪  n| := by
    apply abs_add
  have l3 : |L - a n| = |-(a n - L)| := by
↪  ring_nf
  have l4 : |-(a n - L)| = |(a n - L)| := by
↪  apply abs_neg
  linarith [l1, l2, l3, l4, hN]
```

6.2.4. Prueba en lenguaje natural

Demostración: Como $L \neq 0$, tenemos $|L| > 0$, y por tanto $\varepsilon := |L|/2 > 0$.

Por la convergencia de a hacia L, existe N tal que $|a(n) - L| < |L|/2$ para todo $n \geq N$.

Para cualquier $n \geq N$ tenemos:

$$|L| = |a(n)+(L-a(n))| \leq |a(n)|+|L-a(n)| = |a(n)|+|a(n)-L| <$$

Reordenando:

$$|L| - \frac{|L|}{2} < |a(n)|,$$

por lo que $|a(n)| > |L|/2$, lo que implica $|a(n)| \geq |L|/2$. □

6.2.5. Aplicaciones

Este resultado es esencial para demostrar que el recíproco de una sucesión convergente (con límite no nulo) es a su vez convergente. Garantiza que los denominadores no se aproximan a cero, lo que haría que los recíprocos divergieran.

6.3. Continuidad del valor absoluto: Sucesiones de valores absolutos

La función valor absoluto se comporta de manera muy natural respecto a los límites: si una sucesión converge, la sucesión de los valores absolutos converge al valor absoluto del límite. Esto es una manifestación de la continuidad de la función valor absoluto.

La propiedad clave que usaremos es que el valor absoluto es Lipschitz continuo con constante 1, es decir, $\big||x| - |y|\big| \leq |x - y|$ para todo $x, y \in \mathbb{R}$. Esta desigualdad captura la idea de que el valor absoluto no aumenta las distancias.

6.3.1. Nuevas herramientas

6.3.1.1. Propiedad de Lipschitz: `abs_Lipschitz`

El teorema `abs_Lipschitz` establece que para cualesquiera $x, y \in \mathbb{R}$ se tiene $\big||x| - |y|\big| \leq |x - y|$. A veces se denomina *desigualdad triangular inversa* para valores absolutos.

6.3.2. Enunciado matemático

Teorema 6.3.1. *Si $a : \mathbb{N} \to \mathbb{R}$ converge a L, y $b : \mathbb{N} \to \mathbb{R}$ se define por $b(n) = |a(n)|$ para todo n, entonces b converge a $|L|$.*

6.3.3. Demostración en Lean

```
1  theorem AbsLim (a : ℕ → ℝ) (L : ℝ) (aToL :
   ↪  SeqLim a L)
2     (b : ℕ → ℝ) (bEqAbsa : ∀ n, b n = |a n|) :
3     SeqLim b |L| := by
4   intro ε hε
5   specialize aToL ε hε
```

```
  choose N hN using aToL
  use N
  intro n hn
  specialize hN n hn
  specialize bEqAbsa n
  rewrite [bEqAbsa]
  have : |(|a n|) - (|L|)| ≤ |a n - L| := by
↪  apply abs_Lipschitz
  bound
```

6.3.4. Prueba en lenguaje natural

Demostración: Sea $\varepsilon > 0$ dado. Como $a(n) \to L$, existe N tal que $|a(n) - L| < \varepsilon$ para todo $n \geq N$.

Para cualquier $n \geq N$:

$$|b(n) - |L|| = \big||a(n)| - |L|\big| \leq |a(n) - L| < \varepsilon,$$

donde hemos usado la propiedad de Lipschitz del valor absoluto. Por tanto $b(n) \to |L|$. □

6.3.5. Por qué importa este resultado

Este teorema es un caso especial de un principio mucho más general: las *funciones continuas preservan los límites*. La función valor absoluto es continua en todas partes, por lo que transforma sucesiones convergentes en sucesiones convergentes. Este principio se extiende a todas las funciones continuas y es fundamental en el análisis matemático.

6.4. Recíprocos de sucesiones convergentes

Uno de los teoremas de límites más importantes concierne a los recíprocos: si una sucesión converge a un límite no nulo, entonces la sucesión de los recíprocos converge al recíproco del límite. Este resultado es crucial para demostrar teoremas sobre cocientes y funciones racionales.

La prueba combina varias técnicas que hemos desarrollado: demostrar que la sucesión permanece acotada lejos de cero (de modo que los recíprocos no divergen), manipular cuidadosamente expresiones algebraicas con fracciones, y usar la desigualdad triangular para controlar los términos de error.

6.4.1. Nuevas herramientas

6.4.1.1. Valor absoluto de cocientes: `abs_div`

Para cualesquiera $x, y \in \mathbb{R}$ (con $y \neq 0$) se tiene $|x/y| = |x|/|y|$. Esto permite separar los valores absolutos a través de una división.

6.4.1.2. No nulo a partir de valor absoluto positivo: `nonzero_of_abs_pos`

Si $0 < |x|$, entonces $x \neq 0$. Esto es útil para verificar que la división es válida.

6.4.2. Enunciado matemático

Teorema 6.4.1. *Si $a : \mathbb{N} \to \mathbb{R}$ converge a L con $L \neq 0$, y $b : \mathbb{N} \to \mathbb{R}$ se define por $b(n) = 1/a(n)$, entonces b converge a $1/L$.*

6.4.3. Enfoque estratégico

El reto es que la expresión $|1/a(n) - 1/L|$ involucra recíprocos, que pueden ser difíciles de acotar directamente. Los pasos clave son:

1. Usar el teorema `EventuallyGeHalfLim` para garantizar que $|a(n)| \geq |L|/2$ eventualmente.
2. Elegir épsilon cuidadosamente: usar $\varepsilon \cdot |L|^2/2$ al aplicar la convergencia de a hacia L.
3. Simplificar algebraicamente: $|1/a(n) - 1/L| = |(L - a(n))/(a(n) \cdot L)|$.
4. Separar el valor absoluto usando `abs_div`.
5. Acotar el denominador usando nuestra cota inferior de $|a(n)|$.

6. Usar desigualdades encadenadas para mostrar que el resultado es menor que ε.

6.4.4. Demostración en Lean

```
theorem InvLim (a : ℕ → ℝ) (L : ℝ) (aToL :
↪  SeqLim a L)
    (LneZero : L ≠ 0) (b : ℕ → ℝ)
    (bEqInva : ∀ n, b n = 1 / a n) :
    SeqLim b (1 / L) := by
  choose NhalfL hNhalfL using
    EventuallyGeHalfLim a L aToL LneZero
  intro ε hε
  have : 0 < |L| := by apply abs_pos_of_nonzero
↪  LneZero
  specialize aToL (ε * |L| * |L| / 2) (by bound)
  choose Na hNa using aToL
  use Na + NhalfL
  intro n hn
  specialize bEqInva n
  rewrite [bEqInva]
  have hnHalfL : NhalfL ≤ n := by bound
  have hna    : Na ≤ n      := by bound
  specialize hNhalfL n hnHalfL
  specialize hNa     n hna
  have : 0 < |a n|  := by bound
  have : a n ≠ 0    := by apply
↪  nonzero_of_abs_pos this
  have l1 : |1 / a n - 1 / L| =
      |(L - a n) / (a n * L)| := by field_simp
  have l2 : |(L - a n) / (a n * L)| =
      |L - a n| / |a n * L| := by apply abs_div
  have l3 : |L - a n| / |a n * L| =
```

```
      |L - a n| / (|a n| * |L|) := by bound
  have l4 : |L - a n| = |-(a n - L)| := by
↪  ring_nf
  have l5 : |-(a n - L)| = |a n - L| := by apply
↪  abs_neg
  have l6 : |a n - L| / (|a n| * |L|) <
      (ε * |L| * |L| / 2) / (|a n| * |L|) := by
        field_simp; nlinarith
  have l10 : |L - a n| / (|a n| * |L|) =
      |-(a n - L)| / (|a n| * |L|) := by rw [l4]
  have l11 : |-(a n - L)| / (|a n| * |L|) =
      |a n - L| / (|a n| * |L|) := by rw [l5]
  have l13 : ε * |L| * |L| / 2 / (|a n| * |L|) =
      ε * |L| / 2 / |a n| := by field_simp
  have l14 : ε * |L| / 2 / |a n| ≤ ε := by
        field_simp; bound
  linarith [l1, l2, l3, l10, l11, l6, l13, l14]
```

6.4.5. Prueba en lenguaje natural

Demostración: Sea $\varepsilon > 0$ dado. Como $L \neq 0$, por el teorema `EventuallyGeHalfLim`, existe N_1 tal que $|a(n)| \geq |L|/2$ para todo $n \geq N_1$.

Como $|L| > 0$, tenemos $\varepsilon \cdot |L|^2/2 > 0$. Por la convergencia de a hacia L, existe N_2 tal que $|a(n) - L| < \varepsilon |L|^2/2$ para todo $n \geq N_2$.

Sea $N = N_1 + N_2$. Para cualquier $n \geq N$ se tiene $|a(n)| \geq |L|/2 > 0$, así que $a(n) \neq 0$ y $b(n) = 1/a(n)$ está bien

definido. Entonces:

$$|b(n) - 1/L| = |1/a(n) - 1/L| = \left|\frac{L - a(n)}{a(n) \cdot L}\right| = \frac{|L - a(n)|}{|a(n)| \cdot |L|} = \frac{|a}{|a}$$
$$< \frac{\varepsilon |L|^2/2}{|a(n)| \cdot |L|} \leq \frac{\varepsilon |L|^2/2}{(|L|/2) \cdot |L|} = \frac{\varepsilon |L|^2/2}{|L|^2/2} = \varepsilon.$$

Por tanto $b(n) \to 1/L$. □

6.4.6. Prueba conseguida

Acabas de completar una de las pruebas más desafiantes del análisis elemental. El teorema del límite del recíproco es un hito importante: has demostrado que los recíprocos preservan la convergencia (cuando el límite es no nulo).

6.4.7. Lo que lograste

Esta prueba requirió orquestar múltiples técnicas sofisticadas:

- Usar `EventuallyGeHalfLim` para garantizar que los denominadores permanezcan acotados lejos de cero.
- Elegir un épsilon cuidadosamente calibrado ($\varepsilon \cdot |L|^2/2$) para que el álgebra funcione.
- Manipular expresiones fraccionarias complejas con denominador común.

- Aplicar `abs_div` para separar los valores absolutos a través de la división.
- Encadenar una secuencia de desigualdades para alcanzar la cota final.

Cada paso se apoyó en los anteriores, mostrando cómo las pruebas matemáticas se construyen a partir de bloques cuidadosamente ensamblados.

6.4.8. Aplicaciones y extensiones

Con este teorema en mano, ya disponemos de un conjunto completo de herramientas para los límites de funciones racionales. Combinado con los resultados anteriores sobre sumas y productos, ahora podemos demostrar:

> Si $a(n) \to L$ y $c(n) \to M$ con $M \neq 0$, entonces $a(n)/c(n) \to L/M$.

La prueba es directa: primero se muestra que $1/c(n) \to 1/M$ usando el teorema del recíproco que acabamos de probar, y luego se usa el teorema del producto para mostrar que $a(n) \cdot (1/c(n)) \to L \cdot (1/M) = L/M$.

Esto completa la aritmética fundamental de los límites: sumas, productos y cocientes. Estos son los bloques de construcción para analizar límites de polinomios, funciones racionales y expresiones mucho más complejas a lo largo del cálculo y el análisis.

6.4.9. Dominio de la técnica

El teorema del recíproco ilustra una lección crucial en la demostración matemática: a veces el épsilon «correcto» no es la elección obvia. La expresión $\varepsilon \cdot |L|^2/2$ puede parecer misteriosa al principio, pero está precisamente diseñada para que las desigualdades finales resulten correctas. Este tipo de pensamiento estratégico —trabajar hacia atrás desde lo que necesitas para saber qué debes asumir— está en el corazón de la resolución de problemas matemáticos.

Ya dominas las técnicas esenciales para demostrar teoremas de límites. ¡Bien hecho!

7. Teoremas avanzados de límites e inducción

7.1. Introducción: Inducción Matemática

Simplicio: Oye, Sócrates, he estado pensando en algo que me ha estado molestando. Cuando demostramos cosas en matemáticas, normalmente probamos un enunciado específico. Pero, ¿qué pasa si quiero demostrar que algo es verdad para todos los números naturales? Por ejemplo, ¿cómo demuestro un enunciado para $0, 1, 2, 3, 4$, y así sucesivamente...para siempre?

Sócrates: ¡Ah, una excelente pregunta! Tienes razón en que no podemos simplemente comprobar cada caso uno por uno—eso llevaría un tiempo infinito. Dime, Simplicio, ¿alguna vez has subido una escalera?

Simplicio: ¡Claro que sí! ¿Pero qué tiene eso que ver con esto?

Sócrates: Bueno, imagina una escalera infinitamente alta que llega hasta el cielo. Si quisiera convencerte de que puedes subir a cualquier peldaño de esta escalera, ¿qué tendría que mostrarte?

Simplicio: Mmm...supongo que tendrías que demos-

trarme que puedo alcanzar el peldaño más bajo, ¿no?

Sócrates: ¡Buen comienzo! ¿Y qué más?

Simplicio: Bueno, si estoy parado en cualquier peldaño particular, necesitaría saber que puedo alcanzar el siguiente. Así que, si siempre puedo pasar de un peldaño al siguiente...

Sócrates: ¡Exactamente! Entonces, si puedes alcanzar el primer peldaño, y siempre puedes pasar del peldaño k al peldaño $k + 1$, ¿qué puedes concluir?

Simplicio: ¡Oh! ¡Entonces puedo alcanzar cualquier peldaño que quiera! Si quiero llegar al peldaño 100, simplemente empiezo en el peldaño 0, subo al peldaño 1, luego al 2, y sigo subiendo hasta llegar al 100. ¡Y lo mismo funciona para cualquier número!

Sócrates: ¡Precisamente! Esta es la esencia de la inducción matemática. Para demostrar que algo es verdad para todos los números naturales n, necesitas exactamente dos cosas:

- **Caso base:** demuestra que es verdad para $n = 0$.
- **Paso inductivo:** demuestra que si es verdad para $n = k$, entonces es verdad para $n = k + 1$.

Simplicio: Espera, pero en el paso inductivo, ¿no estamos asumiendo lo que queremos probar? ¿No es eso un razonamiento circular?

Sócrates: ¡Una observación perspicaz! Pero no, no es circular. No estamos asumiendo que el enunciado es

verdad para todo n. Solo estamos asumiendo que es verdad para un valor particular k, y usando esa suposición para demostrar que es verdad para $k+1$. Estamos probando una implicación: "si $P(k)$ entonces $P(k+1)$". Combinado con el caso base, esto crea una reacción en cadena que alcanza cualquier número natural.

Simplicio: Hmm, creo que lo entiendo. Entonces, ¿la suposición "es verdad para k" se llama hipótesis inductiva?

Sócrates: ¡Exactamente! Y esa hipótesis es tu herramienta más poderosa. Es como tener un punto de apoyo en el peldaño k desde el que puedes impulsarte para llegar al peldaño $k+1$.

Simplicio: Pero, ¿por qué funciona esto? Es decir, ¿por qué debo creer en este principio?

Sócrates: ¡Ah, una pregunta profunda! Viene de la propia definición de los números naturales. ¿Cómo crees que se construyen los números naturales?

Simplicio: Bueno...supongo que comenzamos con 0. Y luego tenemos 1, que es $0+1$. Y 2 es $1+1$. Así que cada número es el "sucesor" del anterior.

Sócrates: ¡Perfecto! Los números naturales están definidos precisamente por este proceso:

- Cero es un número natural.
- Si k es un número natural, entonces $k+1$ también lo es.

- Estos son los únicos números naturales.

¿Ves cómo esto refleja la estructura de la inducción?

Simplicio: ¡Oh! El caso base corresponde a "cero es un número natural", y el paso inductivo corresponde a "si k es un número natural, entonces $k+1$ también lo es". ¡La inducción es simplemente la construcción de los números naturales convertida en una técnica de demostración!

Sócrates: ¡Precisamente! No hay "huecos" en los números naturales—ningún número que no pueda alcanzarse partiendo de 0 y sumando 1 repetidamente. Por eso funciona la inducción.

De hecho, esta no es solo una observación filosófica—¡es literalmente cómo se implementan los números naturales en Lean! En la teoría de tipos de Lean, un número natural se define inductivamente como:

- `zero : ℕ`, el caso base, o
- `succ n : ℕ`, el sucesor de otro número natural n.

Así es como se ve esto en el núcleo de Lean:

```
inductive Nat where
  | zero : Nat
  | succ (n : Nat) : Nat
```

Simplemente declaras la existencia de un número natural llamado `zero`, y luego declaras que, dado cualquier número natural *n*, existe otro llamado `succ n`. (La palabra `succ` es solo un nombre; podríamos haberlo llamado `Alice n`. Lo importante es que estamos dando una manera de construir un nuevo número natural a partir de uno ya existente.)

Así, el número 3, por ejemplo, está representado literalmente como `succ (succ (succ zero))`. ¡El principio de inducción no solo se parece a esta construcción, sino que lo explota directamente! Cuando demuestras algo por inducción en Lean, estás trabajando con la estructura computacional real de cómo los números naturales existen en el sistema.

Simplicio: ¡Es increíble! ¿Entonces la inducción no es solo una técnica de demostración, sino que está integrada en el tejido mismo de cómo Lean entiende los números?

Sócrates: Exactamente. El principio de inducción matemática es un teorema en muchos marcos matemáticos, pero en la teoría de tipos es una consecuencia fundamental de cómo están definidos los números naturales.

Simplicio: Bien, estoy convencido de que esta es una técnica de demostración legítima. ¿Puedes darme un ejemplo?

Sócrates: Por supuesto.

7.2. Jefe Final: Recíprocos de Sucesiones Convergentes

Uno de los teoremas de límites más importantes concierne a los recíprocos: si una sucesión converge a un límite no nulo, entonces la sucesión de recíprocos converge al recíproco del límite. Este resultado es crucial para demostrar teoremas sobre cocientes y funciones racionales.

Este es un nivel de *Jefe Final*—requiere sintetizar múltiples técnicas: trabajar con límites no nulos, manipular expresiones algebraicas complejas y elegir cuidadosamente las estrategias con épsilon.

7.2.1. El Marco Matemático

Teorema 7.2.1. *Sea $a : \mathbb{N} \to \mathbb{R}$ una sucesión que converge a L con $L \neq 0$, y sea $b : \mathbb{N} \to \mathbb{R}$ definida por $b(n) = 1/a(n)$ para todo n. Entonces b converge a $1/L$.*

Esta es la demostración técnicamente más desafiante de esta serie de conferencias.

7.2.2. Nuevas Herramientas

abs_div: Para cualesquiera números reales x e y (con $y \neq 0$), se tiene $|x/y| = |x|/|y|$.

nonzero_of_abs_pos: Si $0 < |x|$, entonces $x \neq 0$.

7.2.3. Enfoque Estratégico

Los desafíos clave son:

- Asegurar que $a(n) \neq 0$ eventualmente, para que los recíprocos estén bien definidos.
- Acotar $|1/a(n) - 1/L|$ obteniendo un denominador común.
- Elegir el épsilon correcto al aplicar la convergencia de a hacia L.
- Usar la cota inferior sobre $|a(n)|$ para controlar los recíprocos.

La percepción crítica es que el "épsilon correcto" no es la elección obvia. Usamos $\varepsilon \cdot |L|^2/2$, que está diseñado precisamente para que las desigualdades finales funcionen.

7.2.4. Solución en Lean

```
Statement InvLim (a : ℕ → ℝ) (L : ℝ) (aToL : SeqLim a
↪  L)
    (LneZero : L ≠ 0) (b : ℕ → ℝ)
    (bEqInva : ∀ n, b n = 1 / a n) :
    SeqLim b (1 / L) := by
  choose NhalfL hNhalfL using EventuallyGeHalfLimPos a
↪  L
    aToL LneZero
  intro ε hε
```

```
  have : 0 < |L| := by apply abs_pos_of_nonzero LneZero
  specialize aToL (ε * |L| * |L| / 2) (by bound)
  choose Na hNa using aToL
  use Na + NhalfL
  intro n hn
  specialize bEqInva n
  rewrite [bEqInva]
  have hnHalfL : NhalfL ≤ n := by bound
  have hna : Na ≤ n := by bound
  specialize hNhalfL n hnHalfL
  specialize hNa n hna
  have : 0 < |a n| := by bound
  have : a n ≠ 0 := by apply nonzero_of_abs_pos this
  have l1 : |1 / a n - 1 / L| = |(L - a n) / (a n * L)|
↪  :=
    by field_simp
  have l2 : |(L - a n) / (a n * L)| = |L - a n| / |a n
↪  * L| :=
    by apply abs_div
  have l3 : |L - a n| / |a n * L| = |L - a n| / (|a n|
↪  * |L|) :=
    by bound
  have l4 : |L - a n| = |-(a n - L)| := by ring_nf
  have l5 : |-(a n - L)| = |a n - L| := by apply
↪  abs_neg
  have l6 : |a n - L| / (|a n| * |L|) <
      (ε * |L| * |L| / 2) / (|a n| * |L|) := by
    field_simp; bound
  have l10 : |L - a n| / (|a n| * |L|) =
      |-(a n - L)| / (|a n| * |L|) := by rewrite [l4];
↪  rfl
  have l11 : |-(a n - L)| / (|a n| * |L|) =
      |a n - L| / (|a n| * |L|) := by rewrite [l5]; rfl
  have l13 : ε * |L| * |L| / 2 / (|a n| * |L|) =
      ε * |L| / 2 / |a n| := by field_simp
  have l14 : ε * |L| / 2 / |a n| ≤ ε := by
    field_simp; bound
  linarith [l1, l2, l3, l10, l11, l6, l13, l14]
```

7.2.5. Demostración en Lenguaje Natural

Demostración: Supongamos que $a(n) \to L$ con $L \neq 0$, y sea $b(n) = 1/a(n)$. Debemos demostrar que $b(n) \to 1/L$.

Primero, usando el teorema `EventuallyGeHalfLimPos`, existe N_1 tal que para todo $n \geq N_1$ se tiene $|a(n)| \geq |L|/2 > 0$, lo que garantiza que $a(n) \neq 0$ y $b(n)$ está bien definida.

Dado $\varepsilon > 0$, como $a(n) \to L$, existe N_2 tal que para todo $n \geq N_2$:

$$|a(n) - L| < \frac{\varepsilon |L|^2}{2}.$$

Sea $N = N_1 + N_2$. Para cualquier $n \geq N$:

$$\begin{aligned}
|b(n) - 1/L| &= \left| \frac{1}{a(n)} - \frac{1}{L} \right| \\
&= \left| \frac{L - a(n)}{a(n) \cdot L} \right| \\
&= \frac{|L - a(n)|}{|a(n)| \cdot |L|} \\
&= \frac{|a(n) - L|}{|a(n)| \cdot |L|} \\
&< \frac{\varepsilon |L|^2 / 2}{|a(n)| \cdot |L|} \\
&= \frac{\varepsilon |L|}{2\,|a(n)|} \\
&\leq \frac{\varepsilon |L|}{2 \cdot |L|/2} \\
&= \varepsilon.
\end{aligned}$$

Por lo tanto, $b(n) \to 1/L$. □

7.2.6. Aplicaciones y Extensiones

Con este teorema, ahora disponemos de un conjunto completo de herramientas para los límites de funciones racionales. Combinado con los resultados sobre sumas y productos, podemos demostrar:

Teorema 7.2.2 (Cociente). *Si $a(n) \to L$ y $c(n) \to M$ con $M \neq 0$, entonces $a(n)/c(n) \to L/M$.*

La demostración es directa: primero se muestra que

$1/c(n) \to 1/M$ usando el teorema del recíproco, y luego se aplica el teorema del producto para demostrar que $a(n) \cdot (1/c(n)) \to L \cdot (1/M) = L/M$.

Esto completa la aritmética fundamental de los límites: sumas, productos y cocientes—los bloques de construcción para analizar polinomios, funciones racionales y expresiones complejas a lo largo del cálculo y el análisis.

7.3. Por Casos: Análisis de Casos Sin Hipótesis

Cuando ya tienes una hipótesis `h : P ∨ Q`, puedes usar `cases'` para dividir el objetivo en dos sub-objetivos. Pero, ¿cómo divides en casos cuando no tienes una hipótesis de ese tipo? Ahí es donde entra la táctica `by_cases`.

7.3.1. La Táctica `by_cases`

La táctica `by_cases` tiene la sintaxis `by_cases h : hecho`, donde `h` es el nombre para una nueva hipótesis y `hecho` es el hecho afirmado en ella. Llamar a `by_cases` crea dos sub-objetivos:

- Uno con la hipótesis adicional `h : hecho`.
- Otro con la hipótesis `h : ¬hecho`.

Esto permite realizar un análisis de casos sobre cualquier proposición, no solo aquellas que ya tienes como hipótesis.

7.3.2. Aplicación: Eventualmente Mayor que la Mitad del Límite

Podemos usar `by_cases` para demostrar una versión más general de `EventuallyGeHalfLimPos` que no requiere la suposición $L \neq 0$.

Teorema 7.3.1. *Sea $a : \mathbb{N} \to \mathbb{R}$ una sucesión que converge a L (sin suponer que $L \neq 0$). Entonces existe N tal que para todo $n \geq N$ se tiene $|a(n)| \geq |L|/2$.*

7.3.3. Solución en Lean

```
Statement EventuallyGeHalfLim (a : ℕ → ℝ) (L : ℝ)
    (aToL : SeqLim a L) :
    ∃ N, ∀ n ≥ N, |L| / 2 ≤ |a n| := by
  by_cases h : L = 0
  · use 1
    intro n hn
    rewrite [h]
    bound
  · apply EventuallyGeHalfLimPos a L aToL h
```

7.3.4. Demostración en Lenguaje Natural

Demostración: Procedemos por casos según si $L = 0$ o no.

Caso 1: Si $L = 0$, entonces $|L|/2 = 0$. Como $|a(n)| \geq 0$ para todo n, podemos tomar $N = 1$ y la desigualdad $|L|/2 \leq |a(n)|$ se cumple trivialmente para todo $n \geq N$.

Caso 2: Si $L \neq 0$, podemos aplicar el teorema `EventuallyGe` (que requiere $L \neq 0$ como hipótesis) para obtener el N deseado. □

7.3.5. Importancia

La táctica `by_cases` es esencial para manejar proposiciones cuyo valor de verdad importa pero no se conoce de antemano. Nos permite:

- Manejar casos límite de manera sistemática.
- Demostrar teoremas con hipótesis más débiles.
- Aplicar distintas estrategias de demostración según el caso.

Esta técnica es omnipresente en el razonamiento matemático, desde el análisis hasta el álgebra y la combinatoria.

7.4. Inducción Matemática

La inducción es una de las técnicas de demostración más poderosas para enunciados sobre números naturales. Permite demostrar infinitos enunciados (uno para cada número natural) usando solo dos pasos: un caso base y un paso inductivo.

7.4.1. El Principio de Inducción Matemática

Para demostrar que una propiedad $P(n)$ se cumple para todos los números naturales n, basta con demostrar:

1. **Caso Base:** $P(0)$ es verdadero.
2. **Paso Inductivo:** Para todo k, si $P(k)$ es verdadero, entonces $P(k+1)$ es verdadero.

La intuición es como subir una escalera infinita: si puedes alcanzar el primer peldaño (caso base), y si estar en cualquier peldaño te permite alcanzar el siguiente (paso inductivo), entonces puedes alcanzar todos los peldaños.

7.4.2. La Táctica `induction'`

La sintaxis de la inducción en Lean es: `induction' n with k hk`. Esto significa:

- Aplicar inducción sobre la variable n.
- Usar k como nueva variable muda en el paso inductivo.
- Usar `hk` como hipótesis de inducción sobre k.

7.4.3. Nueva Herramienta: `ge_one_of_nonze`

Si un número natural $n \neq 0$, entonces $1 \leq n$. Este lema sencillo es útil para manejar el caso en que hemos superado el caso base.

7.4.4. Ejemplo: Crecimiento Exponencial

Teorema 7.4.1. *Para todo número natural n, se tiene* $n < 2^n$.

Este teorema captura la idea de que las funciones exponenciales crecen más rápido que las lineales—un hecho fundamental con aplicaciones en toda la matemática y la informática.

7.4.5. Solución en Lean

```
Statement (n : ℕ) : n < 2 ^ n := by
  induction' n with k hk
  · norm_num
  · by_cases hk0 : k = 0
    · rewrite [hk0]
```

```
    norm_num
  · have : 1 ≤ k := by apply ge_one_of_nonzero hk0
    have f1 : k + 1 ≤ 2 * k := by bound
    have f2 : 2 * k < 2 * 2 ^ k := by linarith [hk]
    have f3 : 2 * 2 ^ k = 2 ^ (k + 1) := by ring_nf
    linarith [f1, f2, f3]
```

7.4.6. Demostración en Lenguaje Natural

Demostración: Procedemos por inducción sobre n.

Caso Base ($n = 0$): Tenemos $0 < 2^0 = 1$, lo cual es verdadero.

Paso Inductivo: Supongamos que $k < 2^k$ (hipótesis de inducción). Debemos demostrar que $k + 1 < 2^{k+1}$.

Procedemos por casos según si $k = 0$.

Si $k = 0$, entonces $k + 1 = 1 < 2 = 2^1 = 2^{k+1}$.

Si $k \neq 0$, entonces $k \geq 1$. Por lo tanto:

$$\begin{aligned} k + 1 &\leq 2k && \text{(pues } k \geq 1\text{)} \\ &< 2 \cdot 2^k && \text{(por hipótesis de inducción)} \\ &= 2^{k+1}. \end{aligned}$$

Esto completa la inducción. □

7.4.7. Aplicaciones

La inducción matemática es esencial para demostrar:

- Fórmulas para sumas: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.
- Desigualdades: $n! > 2^n$ para n suficientemente grande.
- Propiedades de divisibilidad: $n^3 - n$ es divisible por 6.
- Definiciones recursivas: números de Fibonacci, factoriales, etc.
- Corrección de algoritmos: demostrar invariantes de bucles.

La combinación de la inducción con otras técnicas de demostración (como el análisis de casos) crea un poderoso conjunto de herramientas para las matemáticas discretas y la informática.

7.4.8. Idea Clave

Esta demostración ilustra una estrategia importante: dentro de una demostración inductiva, puede ser necesario realizar análisis de casos adicionales (usando `by_cases`). La hipótesis de inducción te da ventaja, pero a menudo hay que ser ingenioso en cómo aplicarla, especialmente cuando se tratan casos frontera o cuando el álgebra requiere restricciones adicionales.